Unified Transformer-Based Spatio-Temporal Forecasting of Air Quality and Urban Noise Risk with a Rule-Based Public Advisory Layer

A PROJECT REPORT

Submitted by

SHOVIK BANERJEE [RA2211003010270]
KAUSHANI CHAKRABORTY NAME [RA2211003010227]

Under the Guidance of

Dr. Akella S. Narasimha Raju
(Assistant Professor, C.Tech)

in partial fulfillment of the requirementsfor the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

Department of Computing Technologies
SRM Institute of Science & Technology
Own Work Declaration Form

To be completed by the student for all assessments

Degree/ Course : B.Tech / Computer Science and Technology

Student Name : Shovik Banerjee, Kaushani Chakraborty

Registration Number : RA2211003010270, RA2211003010227

Title of Work : Unified Transformer-Based Spatio-Temporal Forecasting of Air Quality and Urban Noise Risk with a Rule-Based Public Advisory Layer

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:	
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.	
Shovik Banerjee (RA2211003010270)	Kaushani Chakraborty (RA2211003010227)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP401L – Major Project report titled "**Unified Transformer-Based Spatio-Temporal Forecasting of Air Quality and Urban Noise Risk with a Rule-Based Public Advisory Layer**" is the Bonafide work of "**Shovik Banerjee [RA2211003010270], Kaushani Chakraborty [RA2211003010227]**" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Dr Akella S Narasimha Raju	**Dr Niranjana G**
SUPERVISOR Assistant Professor DEPARTMENT OF COMPUTING TECHNOLOGIES	**PROFESSOR &HEAD** DEPARTMENT OF COMPUTING TECHNOLOGIES

EXAMINER 1

EXAMINER 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, Dr Niranjana G, Professor, C.Tech, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr Suresh Anand M, Dr B Prakash** Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr Akella S. Narasimha Raju**, Department of Computing Technologies, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his / her mentorship. He / She provided us with the freedom and support to explore the research topics of our interest. His / Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

ABSTRACT

The accelerated pace of urbanization increases air pollution and environmental noise, resulting in a set of interrelated exposure issues that require the development of forecasting systems with the ability to provide accurate predictions and public advisories. The current literature often considers air quality index (AQI) and noise pollution separately; noise pollution is modeled as a continuous regression task despite its event-oriented nature and low monthly autocorrelation, making it less useful for decision-making. We propose an attention-driven spatio-temporal model that combines government-provided AQI data with diurnal and nocturnal noise pollution measurements into a single monthly dataset, extends it with lagged and rolling features, and compares Linear Regression, XGBoost, and convolutional neural network (CNN) models with several transformer models: a standard Transformer (ReLU/GELU activation functions), a Hybrid Convolution-Attention Transformer, and a simplified Temporal Fusion Transformer. The AQI is predicted using regression analysis, while noise pollution is recast as a risk classification problem to provide probabilities amenable to decision-making. An inference rule layer is used to translate predicted AQI values and noise risk scores into actionable warnings for both daytime and nighttime scenarios. The results show that transformer models provide a consistent boost to AQI prediction accuracy, with the Hybrid Convolution-Attention Transformer providing $R^2 = 0.91$ and MAE ≈ 8.7 on the combined dataset, outperforming non-attention and traditional models. In contrast, direct noise pollution regression is plagued by model-wide instability; recasting noise pollution as a classification problem improves discriminative accuracy, reaching an AUROC of ≈ 0.63 for nighttime scenarios and achieving competitive probabilistic errors (log loss/Brier score). The system provides a realistic exposure guidance framework for city-scale scenarios by integrating transformer-based predictions with decision-oriented risk outputs.

Keywords: Air Quality Index Forecasting; Urban Noise Risk; Spatio-Temporal Transformers; Hybrid Convolution–Attention; Rule-Based Public Advisory

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

• **AQI:** Air Quality Index

• **AUC-PR:** Area Under the Precision–Recall Curve

• **AUROC:** Area Under the Receiver Operati Characteristic Curve

• **CNN:** Convolutional Neural Network

• **CPCB:** Central Pollution Control Board

• **GELU:** Gaussian Error Linear Unit

• **MAE:** Mean Absolute Error

• **MAPE:** Mean Absolute Percentage Error

• **MCC:** Matthews Correlation Coefficient

• **MSE:** Mean Squared Error

• **RMSE:** Root Mean Squared Error

• **RMSLE:** Root Mean Squared Logarithmic Error

• **SHAP**: SHapley Additive exPlanations

• **SMAPE:** Symmetric Mean Absolute Percentage Error

• **TFT:** Temporal Fusion Transformer

• **XAI:** Explainable Artificial Intelligence

• **XGBoost:** Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

We have identified air pollution and environmental noise as two stressors that have been exacerbated by modern urbanization and industrialization. The World Health Organization estimates that the attributable risk of household and ambient air pollution leads to 6.7 million excess deaths each year. Many these excess deaths occur in populations exposed to traffic-related gases and fine particulate matter through cardiopulmonary routes. [1], [3] According to the Health Effects Institute and the State of Global Air initiative, air pollution led to 8.1 million deaths in 2021, stressing that air pollution in urban areas is not only an environmental problem but also a significant health risk. The problem of air pollution, as indicated by air quality scores, is particularly significant in urban areas where air pollution is further aggravated by transportation and industrial emissions. The problem of air pollution, as evidenced by air quality values, is particularly acute in urban areas, where air pollution is caused by transportation and industrial emissions. Environmental noise is now recognized as a serious health problem, similar in importance to air pollution, due to its large non-auditory health effects.

Besides air pollution, environmental noise is gradually recognized as a significant public health risk with significant non-auditory effects. The WHO Regional Office for Europe summarizes the evidence regarding the effect of chronic exposure to environmental noise on outcomes such as sleep disturbance, cardiovascular and metabolic disease, cognitive impairment, and mental health and well-being. [4] In keeping with WHO guidance, there is a general appreciation that excessive noise exposure increases the risk of ischemic heart disease and hypertension and is associated with adverse outcomes independent of annoyance. [5] Importantly, urban noise patterns are often event-related (traffic bursts, construction phases, rhythms of socio-economic activity) with low persistence on a coarse timescale; there is mechanistic and clinical evidence that nocturnal noise exposure can affect sleep architecture and trigger a detectable stress response. As such, decision-support modeling should be structured around risk strata rather than regression on a continuous variable. [6]

From a modelling viewpoint, the conventional air quality forecasting chain has conventionally depended on statistical baselines and tree-based machine learning models (e.g. XGBoost), which are known to work well in stationary environments but are often expected to be less robust under conditions of non-stationarity, covariate heterogeneity, and cross-city generalization. [7] CNN-based models improve short-term feature learning, but they can still be insufficient for long-term dependency modelling in spatio-temporal environmental phenomena—exactly what the transformer attention mechanism is best suited for. Recent studies on air quality forecasting have started to leverage transformer modules for time series forecasting, such as transformer hybrids in Scientific Reports (2024) and multi-granularity spatiotemporal fusion transformers in Information Fusion (2025), which indicate the increasing trend of attention-based air quality forecasting. [8], [9] Nevertheless, the comprehensive modelling of both AQI forecasting and urban noise as a dynamic risk signal, and addressing the modelling challenges of temporal resolution mismatch and autocorrelation in noise, is still an open problem. Based on the original transformer model formulation introduced by Ashish Vaswani et al. and using interpretable multi-horizon models like Temporal Fusion Transformers, this paper proposes an attention-based framework that models AQI forecasting as a regression problem, models noise as a risk classification problem, and closes the loop with a rule-based public recommender that converts forecasts of severity (and sources) into actionable mitigation advice. [10], [11]

1.2 Problem Statement

Urban environments face the dual challenge of rising air pollution and environmental noise, both of which pose serious health risks but are typically studied and modeled in isolation. Existing air quality forecasting systems primarily focus on AQI prediction using statistical or machine learning approaches, often lacking robustness under non-stationary and heterogeneous urban conditions. At the same time, noise pollution is frequently treated as a continuous regression problem despite its event-driven nature and weak temporal autocorrelation, leading to unreliable predictions and limited decision-making value. Furthermore, current systems rarely integrate multi-source data such as meteorological, spatial, and temporal factors into a unified framework. Another key limitation is the absence of actionable outputs, as most models stop at prediction metrics without translating results into meaningful public advisories. Therefore, there is a need for a unified, spatio-temporal, and decision-oriented system that accurately forecasts AQI, models noise as a risk classification problem, and provides interpretable, actionable insights for urban environmental management.

1.3 Motivation

The growing impact of urbanization has intensified exposure to both air pollution and environmental noise, creating a combined public health burden that is not adequately addressed by existing analytical systems. While significant progress has been made in air quality forecasting, most approaches focus solely on AQI and overlook the parallel risk posed by urban noise. At the same time, noise pollution is often poorly modeled due to its event-driven nature, resulting in low predictive reliability when treated as a continuous variable. This disconnect highlights a deeper issue: current environmental intelligence systems are fragmented, lack contextual awareness, and fail to reflect real-world urban dynamics.

Another key motivation lies in the gap between predictive modeling and practical usability. Most existing models prioritize accuracy metrics but do not translate outputs into actionable insights that can support public decision-making or policy interventions. Additionally, traditional and deep learning models struggle to capture long-range temporal dependencies and interactions across heterogeneous data sources such as weather, spatial factors, and seasonal patterns.

This research is motivated by the need to bridge these gaps by developing a unified, attention-driven framework that integrates AQI forecasting with noise risk assessment. The goal is to move beyond prediction toward interpretable, decision-oriented environmental intelligence that can support real-world urban health and planning systems.

1.4 Sustainable Development Goal of the Project

This project aligns strongly with multiple United Nations Sustainable Development Goals (SDGs), as it addresses critical urban environmental and public health challenges through data-driven intelligence and decision support.

SDG 3: Good Health and Well-being

The project directly contributes to improving public health by forecasting air pollution (AQI) and assessing environmental noise risk. By providing early warnings and actionable advisories, it helps reduce exposure to harmful pollutants and excessive noise, thereby mitigating risks related to respiratory diseases, cardiovascular conditions, and sleep disorders.

SDG 11: Sustainable Cities and Communities

Urban sustainability is a core focus of this work. The system enables smarter environmental monitoring and supports city-level decision-making by integrating air quality and noise risk into a unified framework. This helps urban planners and authorities design healthier, safer, and more resilient cities.

SDG 13: Climate Action

Although indirectly, the project contributes to climate action by promoting awareness of pollution patterns and enabling data-driven interventions. Improved monitoring and forecasting can support policies aimed at reducing emissions and managing environmental risks.

SDG 9: Industry, Innovation, and Infrastructure

The use of advanced transformer-based models and explainable AI techniques demonstrates innovation in environmental monitoring systems. The framework can be integrated into smart city infrastructure, enhancing technological capabilities for sustainable development.

Overall, the project supports a transition from passive environmental monitoring to proactive, intelligent, and actionable urban risk management.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of the Research Area

Air quality forecasting has evolved significantly from traditional statistical approaches to advanced machine learning and deep learning techniques. Early methods primarily relied on regression and ensemble models, while recent advancements focus on spatio-temporal learning to capture complex environmental dynamics. With the emergence of transformer architectures, attention-based models have shown strong capability in modeling long-range temporal dependencies. However, most research remains limited to air quality alone, with minimal integration of other critical urban factors such as environmental noise. Additionally, many studies treat pollution prediction as either a static classification or a single-modality forecasting problem, limiting their real-world applicability.

2.2 Existing Models and Frameworks

Several approaches have been explored in prior work. Traditional machine learning models such as Decision Trees, Support Vector Machines, and Random Forests have been widely used for AQI prediction, achieving moderate performance (around 74% accuracy) but failing to capture temporal dependencies effectively [13]. City-specific models, such as Random Forest-based APSI forecasting, have demonstrated high accuracy (~95%) but lack generalizability and temporal awareness [12].

Deep learning approaches, including CNNs and graph-based models, improve short-term forecasting by capturing spatial relationships, achieving high performance (e.g., $R^2 \approx 0.949$) [14]. However, these models are often limited to small datasets and focus only on pollutant regression. More advanced transformer-based models, such as AirFormer and ST-iTransformer, introduce attention mechanisms for capturing long-range dependencies and handling non-stationarity, achieving strong predictive performance [16], [17]. Despite this progress, these frameworks remain largely air-quality-centric and do not incorporate noise pollution or multi-source urban data.

Data infrastructures such as CPCB and ENVIS provide extensive environmental datasets, including potential noise data, but they are rarely integrated into a unified modeling framework [18], [19].

2.3 Limitations Identified from Literature

The literature reveals several key limitations. First, most models focus solely on air quality and ignore environmental noise, despite its growing public health impact. Second, many approaches treat time-series data as static, leading to poor modeling of seasonal and long-term dependencies. Third, noise pollution is often incorrectly modeled as a regression problem, even though it exhibits event-driven behavior with weak temporal autocorrelation [20], [21]. Fourth, existing models lack multi-modal integration, failing to combine air quality, weather, spatial, and noise data into a single framework. Additionally, most studies are geographically constrained and lack cross-city generalization. Finally, there is a significant gap between prediction and practical application, as few systems provide interpretable outputs or actionable decision support.

2.4 Research Objectives

To address these gaps, this research aims to develop a unified spatio-temporal framework that integrates air quality forecasting and noise risk assessment within a single pipeline. The objectives include modeling AQI as a regression problem while reformulating noise as a probabilistic risk classification task, leveraging transformer-based architectures for capturing long-range dependencies, and incorporating multi-source data such as meteorological and spatial features. Furthermore, the study aims to enhance interpretability using explainable AI techniques and bridge the gap between prediction and real-world usability by developing a rule-based advisory system that translates model outputs into actionable public alerts.

2.5 Technologies Utilized

This research uses a spatio-temporal modeling toolkit that is attention-driven and Transformer-based (ranging from the Vanilla Transformer with ReLU/GELU activation functions to the Hybrid Convolution-Attention Transformer and a Temporal Fusion Transformer-inspired model architecture). The model is fine-tuned against machine learning benchmarks (Linear Regression, Random Forest, SVM, XGBoost) and a CNN model for sequence learning with long-range dependencies. The data processing pipeline involves spatio-temporal harmonization, lag/rolling feature construction, and, where applicable, geographical/contextual embeddings. Air Quality Index (AQI) is formulated as a regression forecasting task, while noise is formulated as a day/night risk classification problem with probabilistic assessment metrics (such as AUROC, log loss, or Brier score). The final decision component includes an auditable rule-based recommender system that converts forecasted AQI and noise risk probabilities into actionable public warnings, with interpretability facilitated through attention-explained and feature importance analyses.

Table 1. **This has the extensive research survey of literature.**

S.No	Ref.	First Author	Year	Methodology	Dataset	Performance (reported)	Key strengths	Key limitations	Research gap to close
1	[12]	Richie Muljana	2023	Random Forest classifier + feature importance for APSI	Jakarta Open Data (single city, single year)	~95% accuracy (classification)	Strong baseline; interpretable pollutant ranking	No temporal dependency modelling; single-city/year; no meteo/noise	Sequence learning + cross-city generalisation + multimodal fusion

									(air+noise+weather)
2	[13]	K. M. O. V. K. Kekulunadara	2021	DT / SVM / RF comparison for AQI	Multi-city India hourly AQI (100k+ records)	~74% max accuracy	Large-scale baseline comparison	Treats time-series as static; no DL/transformers; no meteo/noise	Long-range temporal modelling + better feature context + decision-grade outputs
3	[14]	Hong Cheng	2024	STSGCN multi-step PM2.5 forecasting	8 stations, Wuhan (2018–2022)	$R^2 \approx 0.949$; RMSE≈3.39	Explicit spatio-temporal dependency modelling	Limited stations; mainly short-term; PM2.5-only; no noise/source	Multi-factor urban intelligence (AQI + noise risk + source attribution) + broader generalisation
4	[15]	Priyanshi Kotlia	2024	RF & XGBoost AQI classification	Uttarakhand districts	RF≈96%, XGB≈95% accuracy	High accuracy; pollutant importance	Static classification; no forecasting; limited geography	Forecasting + geospatial embeddings + cross-region transfer
5	[16]	Yuxuan Liang	2023	Transformer (AirFormer) with dynamic spatio-temporal attention	1,085 AQ + 2,575 meteo stations (China, 2015–2018)	$R^2 \approx 0.949$; MAE≈16.03	Long-range modelling; attention explainability	No noise integration; country-specific training	Unified air+noise modelling + cross-city/cross-country robustness + action mapping
6	[17]	Rui Zhang	2025	ST-iTransformer (inverted attention + geo embeddings)	Beijing multi-site (UCI), 12 stations	High accuracy (paper-reported)	Targets non-stationarity; spatio-temporal embeddings	No noise/traffic/industry; scalability/real-time validation limited	Practical deployability + multimodal drivers + calibrated decision support
7	[18]	Central Pollution Control Board	—	National AQI / monitoring portals (CAAQMS/NAMP access points)	India-wide monitoring network	—	Authoritative operational data access	Not directly model-ready; heterogeneity/missingness	Harmonised data engineering + resolution alignment for unified modelling
8	[19]	ENVIS/CPCB-linked repositories	—	Environmental information repositories	India environmental data ecosystems	—	Supports pollutant + (often) noise datasets	Fragmented, underused in unified models	Joint learning across air+noise with consistent

									schema & governance
9	[20]	Haizhen Zhang	2011	Environmental noise measurement + evaluation	Campus setting (Southwest Forestry Univ.)	—	Highlights measurement protocol + day/night variation	Not predictive; local study	Noise as dynamic risk signal; classification framing; integration with AQI forecasts
10	[21]	Jun Zhang	2010	Roadside construction noise analysis	Urban road near construction	—	Event-driven noise behaviour evidence	Not predictive; context-specific	Event-driven noise modelling + risk thresholds for public recommendations
11	[22]	Junzhong Ji	2025	Spatio-temporal transformer for forecasting (cross-domain evidence)	Weather forecasting benchmarks	Strong forecasting (journal-reported)	Validates attention for long-range spatio-temporal learning	Different domain (weather)	Transfer spatio-temporal transformer design to air+noise unified forecasting

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

The project was executed using an agile sprint-based methodology to ensure systematic development, iterative validation, and continuous improvement. The entire workflow was divided into three major sprints, each addressing a critical stage of the system: data preparation, model development, and deployment with decision support. This structured approach enabled the team to progressively refine the system, validate intermediate outputs, and adapt to challenges encountered during implementation.

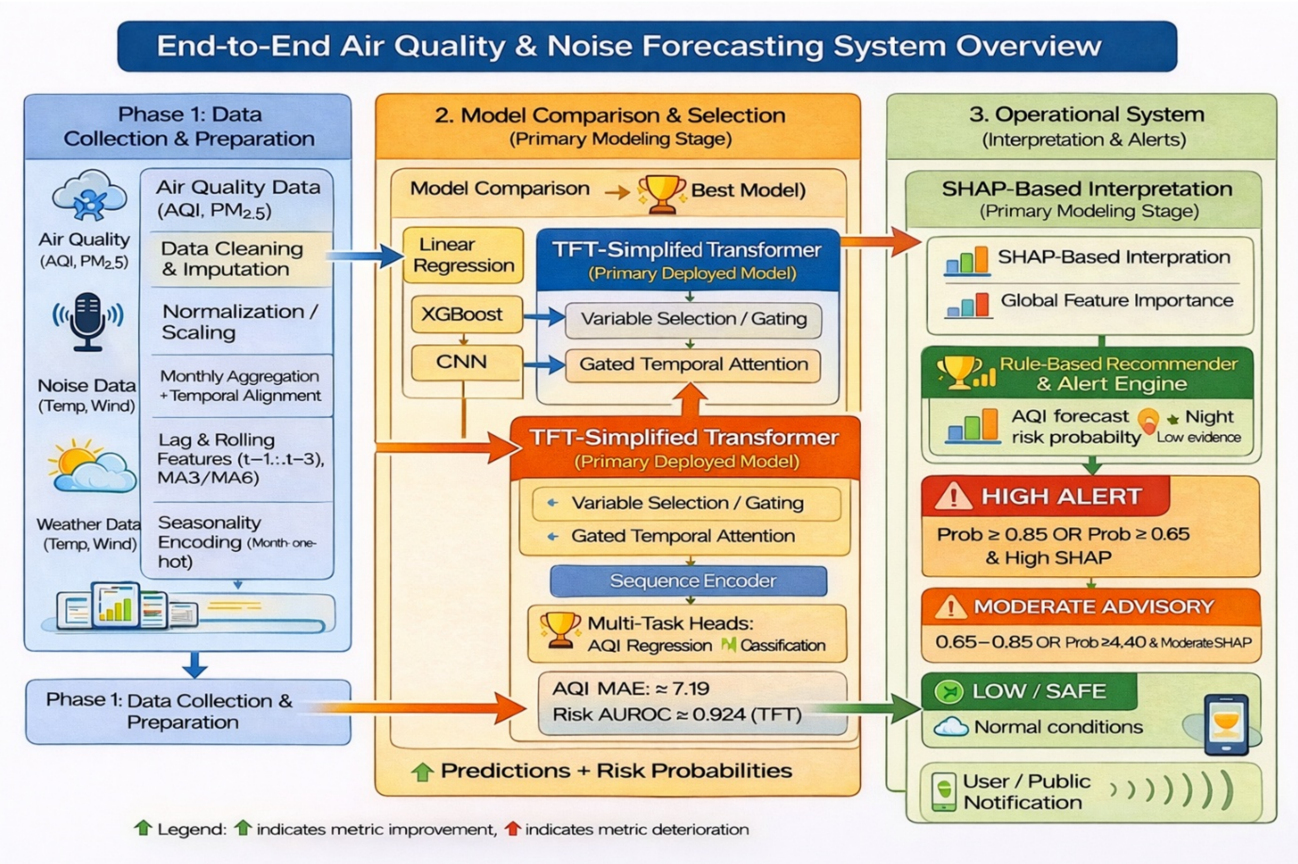


Figure 1 Architecture Diagram: Air Quality and Noise Forecasting System

3.1 SPRINT I: Data Collection, Integration, and Preprocessing

3.1.1 Objectives with user stories of Sprint I

Sprint I primarily focused on establishing a reliable, clean, and consistent data foundation for the overall system. The main objective was to collect air quality (AQI) data and environmental noise datasets from publicly available sources and integrate them into a unified and structured format suitable for further analysis and modeling. This phase was critical because the quality of data directly influences the performance and reliability of predictive models.

From a user perspective, the goal was to ensure that the dataset is synchronized, free from inconsistencies, and enriched with meaningful features. The data scientist required a stable and well-prepared dataset to perform modeling without preprocessing overhead. The system developer needed harmonized multi-modal inputs combining AQI, noise, and contextual features for seamless pipeline integration. Additionally, the researcher aimed to extract meaningful temporal patterns through feature engineering, such as trends and seasonality, to support accurate forecasting. Overall, Sprint I ensured that all stakeholders could work with a unified and dependable data layer.

3.1.2 Functional Document

This sprint involved several critical preprocessing and transformation steps to improve data quality and usability. Missing values were handled using statistical imputation techniques to maintain data continuity and avoid information loss. Outliers were identified and removed using statistical methods to reduce noise while preserving meaningful extreme environmental conditions. Feature normalization was applied to ensure consistent scaling across all variables, which is essential for stable model training.

A key challenge addressed during this phase was the mismatch in temporal resolution between AQI data, which was available at hourly or daily levels, and noise data, which was aggregated monthly. This issue was resolved by converting AQI data into monthly aggregates, ensuring proper alignment between both datasets. Furthermore, advanced feature engineering techniques were implemented, including the creation of lag features ($t-1$ to $t-3$) to capture temporal dependencies, rolling averages (3-month and 6-month) to smooth trends, and seasonal encoding to represent periodic environmental patterns. These enhancements significantly improved the dataset's ability to support temporal learning.

3.1.3 Architecture Document

From an architectural perspective, the system was designed as a modular data processing pipeline. The first layer consisted of a data ingestion component responsible for collecting inputs from multiple sources, including AQI datasets, noise pollution data, and contextual environmental variables. This was followed by a preprocessing layer that handled data cleaning, missing value treatment, outlier removal, and normalization to ensure data consistency.

The next stage involved a feature engineering layer, where temporal and statistical features were generated to enrich the dataset. This included lag variables, rolling statistics, and seasonal indicators. The final output of this pipeline was a structured spatio-temporal dataset that integrates multiple data modalities and is optimized for downstream modeling tasks. This modular design ensured scalability, reusability, and consistency across different stages of the project.

3.1.4 Outcome of objectives / Result Analysis

The outcome of Sprint I demonstrated the successful integration of heterogeneous datasets into a unified and analysis-ready framework. The processed dataset enabled consistent modeling across multiple cities and time periods. Exploratory analysis revealed strong temporal persistence in AQI variables, validating the suitability of regression-based forecasting approaches. In contrast, noise variables exhibited weak correlation and irregular patterns, indicating that they follow an event-driven behavior and may require alternative modeling strategies.

The retrospective analysis highlighted that while data integration and feature engineering were effective, challenges such as temporal misalignment, missing values, and differences in data granularity required careful handling. These challenges were successfully mitigated through aggregation and preprocessing techniques. Moving forward, potential improvements include automating the preprocessing pipeline, enhancing data validation procedures, and incorporating additional contextual features to further improve model performance and robustness.

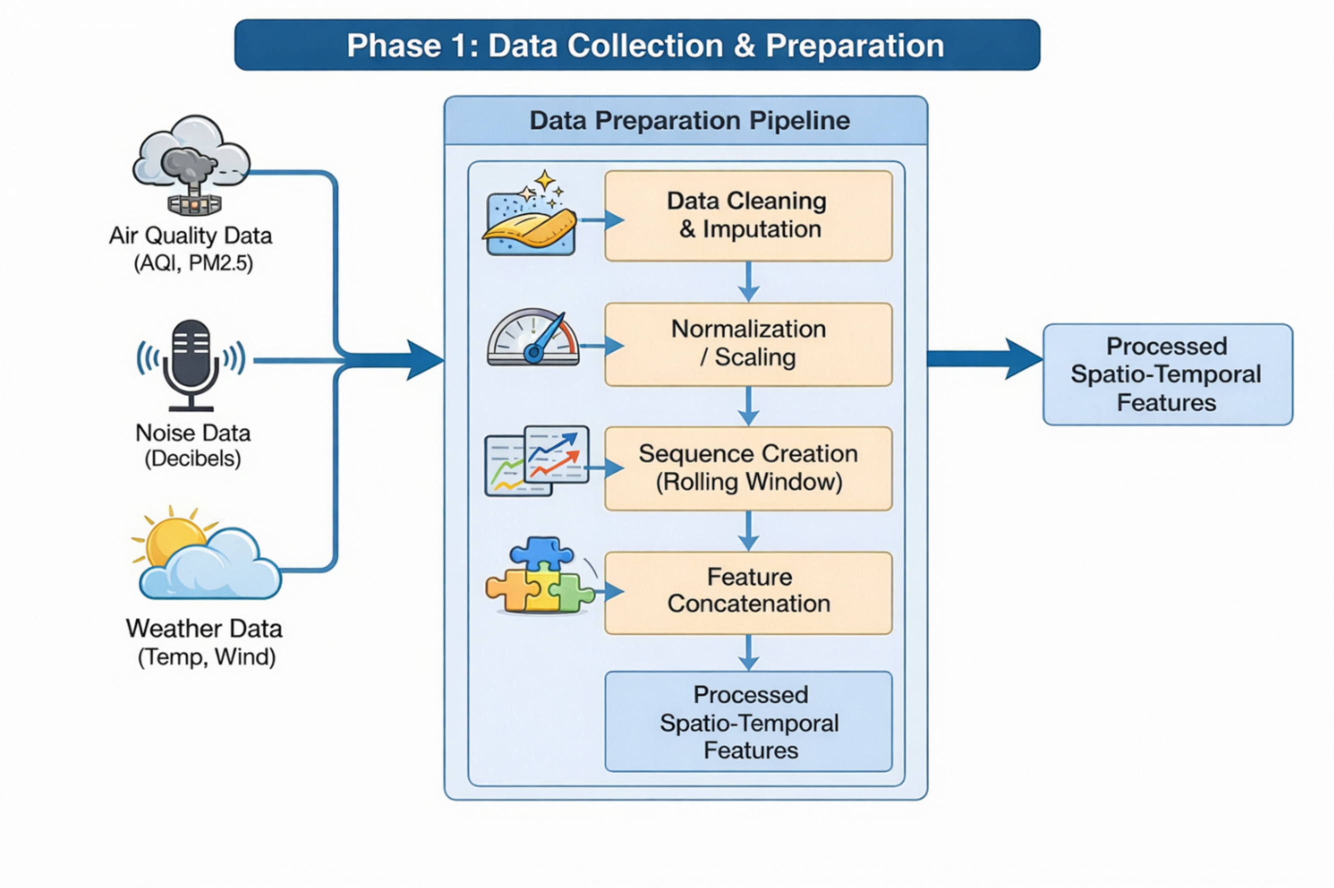


Figure 2: Air and Noise Data Collection and Preparation

3.2 SPRINT II: Model Development and Comparative Analysis

3.2.1 Objectives with user stories of Sprint II

Sprint II focused on the development, evaluation, and selection of predictive models for urban air quality and noise pollution forecasting. The primary objective was to implement a diverse set of baseline and advanced models and systematically evaluate their performance under a consistent experimental setup. This phase aimed to identify the most suitable modeling approach for capturing spatio-temporal patterns in the dataset.

From a user perspective, the goal was to enable researchers to compare multiple model architectures in a fair and standardized manner, ensuring reproducibility and reliability of results. Data scientists required a framework that supports consistent evaluation across models, while system developers needed a clear understanding of model behavior for integration into the pipeline. Additionally, this sprint aimed to improve the representation of noise prediction by reformulating it into a more meaningful and practical form based on observed data limitations.

3.2.2 Functional Document

This sprint involved the implementation of both traditional machine learning and deep learning models. Baseline models such as Linear Regression and XGBoost were developed to establish performance benchmarks. In parallel, deep learning approaches, including Convolutional Neural Networks (CNNs) and transformer-based architectures, were implemented to capture temporal dependencies more effectively.

The transformer-based models included a Vanilla Transformer, a Hybrid Convolution–Attention Transformer, and a TFT-Simplified model. All models were trained using the same preprocessed and lag-enhanced dataset to ensure fair comparison. Evaluation metrics were selected based on task requirements: regression metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2) were used for AQI forecasting, while classification metrics such as accuracy, F1-score, AUROC, and log loss were used for noise risk prediction.

3.2.3 Architecture Document

The architectural design for this sprint consisted of a unified modeling framework. A common input layer received time-series sequences generated from the processed dataset, including lagged and temporal features. These inputs were then passed through multiple parallel model pipelines representing different algorithm families, including linear models, tree-based models, CNNs, and transformer-based architectures.

Each model independently processed the same input data, ensuring consistency in training and evaluation. The output layer generated two types of predictions: continuous AQI values for regression tasks and categorical noise risk levels for classification tasks. This modular design allowed direct comparison of model performance while maintaining scalability and flexibility in experimentation.

3.2.4 Outcome of objectives / Result Analysis

The results of Sprint II provided important insights into the behavior of different models across tasks. AQI forecasting achieved consistently strong performance across all model families due to the presence of clear temporal patterns and autocorrelation in air quality data. In contrast, noise prediction as a regression task failed across all models, with near-zero or negative explanatory power, highlighting the event-driven and weakly structured nature of noise data.

This observation led to a key design decision to reformulate noise prediction as a classification problem, significantly improving reliability and interpretability. Among the evaluated models, the Hybrid Convolution–Attention Transformer achieved the best performance for AQI forecasting by effectively capturing both local and long-range temporal dependencies. The TFT-Simplified model demonstrated the most balanced performance across both AQI and noise tasks, particularly in handling classification reliability.

3.2.5 Sprint Retrospective

The retrospective analysis indicated that the implementation and comparison of multiple models were successfully achieved, providing a clear understanding of their strengths and limitations. However, several challenges were identified, particularly related to class imbalance in noise data and instability in regression-based noise predictions.

It was observed that relying solely on accuracy was insufficient for evaluating classification performance in imbalanced scenarios. As a result, greater emphasis was placed on probabilistic and recall-based metrics. The retrospective also highlighted the need for a more decision-oriented modeling approach, where model outputs are aligned with real-world use cases such as risk detection and alert generation. Future improvements include enhancing class balancing strategies, refining evaluation techniques, and further optimizing model architectures for better generalization.

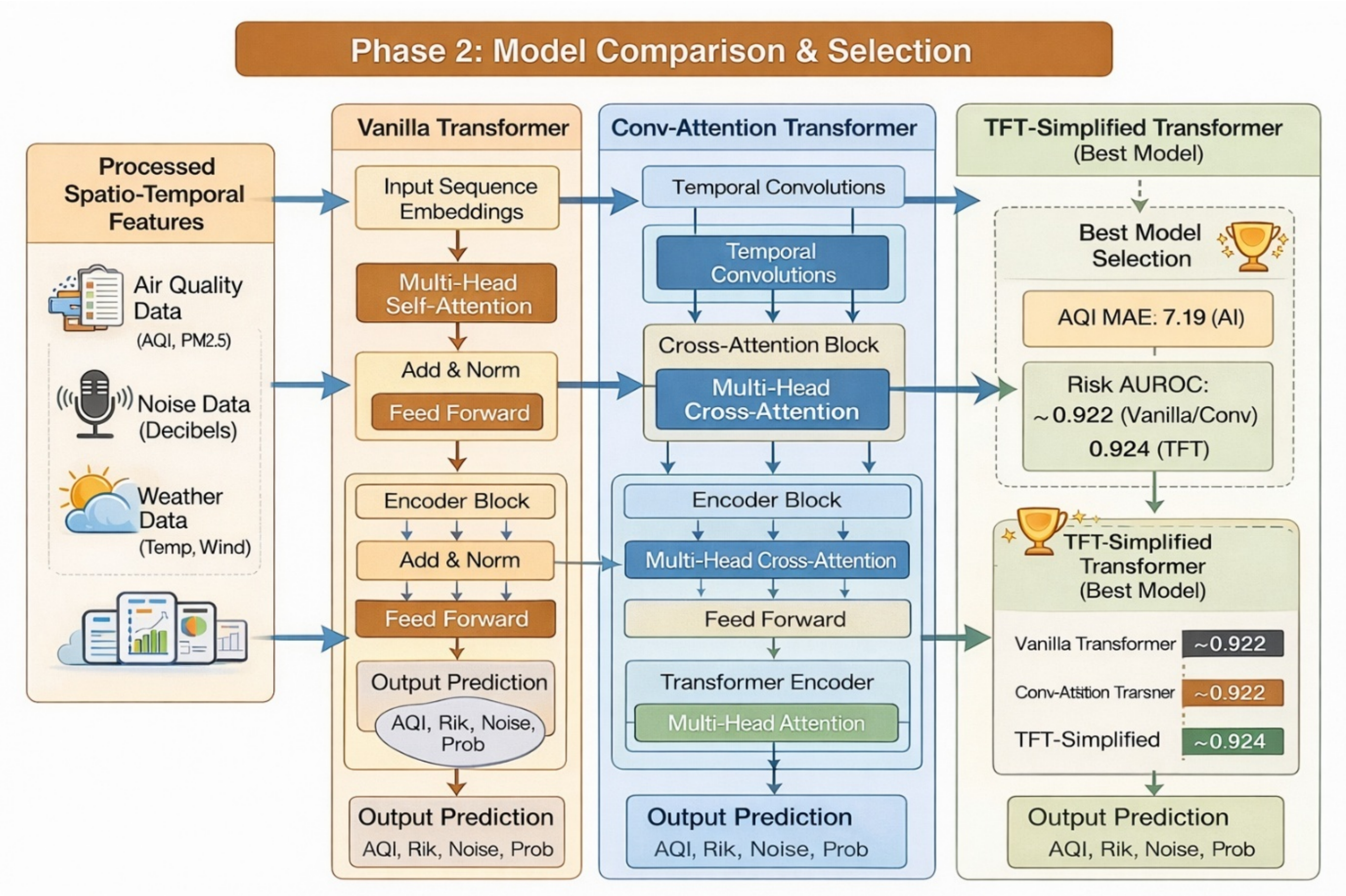


Figure 3: Model Comparison and Selection

3.3 SPRINT III: Deployment, Explainability, and Decision Support

3.2.1 Objectives with user stories of Sprint III

Sprint III focused on converting the predictive modeling framework into a practical decision-support system. The primary objective was to deploy the selected TFT-Simplified model and extend it beyond prediction by integrating explainability and actionable intelligence. This phase emphasized usability, ensuring that model outputs are not only accurate but also interpretable and useful in real-world scenarios.

From a user perspective, policymakers required clear and interpretable insights for decision-making, while end users needed simple and actionable alerts. Additionally, researchers aimed to ensure transparency in model behavior, enabling trust in predictions and understanding of contributing factors. This sprint ensured that the system could bridge the gap between technical outputs and real-world application.

3.2.2 Functional Document

Functionally, this sprint involved processing model outputs, including AQI predictions and noise risk probabilities, through an explainability layer using SHAP (SHapley Additive exPlanations). This allowed both global interpretation (feature importance across the dataset) and local interpretation (feature contribution for individual predictions), improving model transparency.

The interpreted outputs were then passed to a rule-based recommendation system. This system mapped predictions into predefined alert categories such as High Risk, Moderate Advisory, and Low/Safe. Each category was associated with specific recommended actions, enabling users to respond appropriately to environmental conditions. This transformation from raw predictions to structured alerts significantly improved system usability.

3.2.3 Architecture Document

The system architecture at this stage included a prediction layer (TFT-Simplified model), an explainability layer (SHAP), a decision layer (rule-based engine), and an output layer generating alerts and recommendations. This pipeline ensured that the system moved beyond raw predictions to actionable intelligence.

3.2.4 Outcome of objectives / Result Analysis

The outcomes of Sprint III demonstrated successful integration of prediction, explanation, and decision-making into a unified system. The model showed reliable performance, particularly in identifying high-risk noise conditions, especially during nighttime scenarios. The addition of explainability enhanced trust in the system by providing insights into model decisions. Furthermore, the rule-based recommendation system enabled practical usability by translating predictions into actionable alerts, making the system suitable for real-world deployment.

3.2.5 Sprint Retrospective

The retrospective analysis confirmed that the system effectively achieved its objective of becoming a decision-support solution. However, certain challenges were identified, particularly in selecting optimal thresholds for alert generation and calibrating probabilistic outputs for consistent performance. Future improvements include refining threshold tuning strategies, enhancing uncertainty estimation, and extending the system for real-time deployment. These enhancements will further improve reliability, scalability, and applicability in dynamic urban environments.

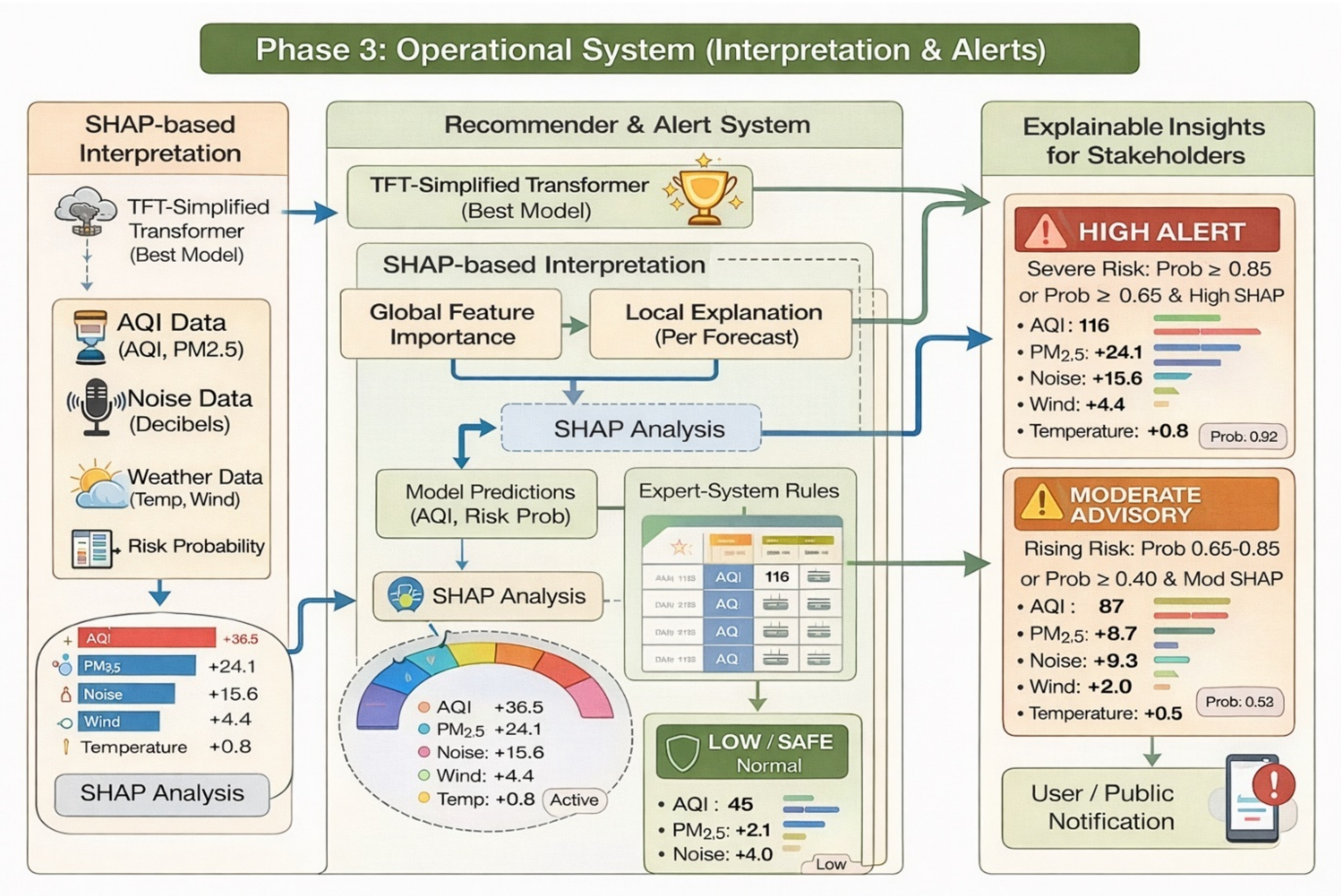


Figure 4: Operational System-Recommender and Alert Systems with XAI

3.4 Flow Diagram

The holistic system flow diagram in Figure 5 consists of a unified decision-grade pipeline that combines diverse urban exposure signals. Phase-1 takes in air quality data (AQI/pollutants), day/night noise levels, and weather feeds, followed by cleaning, imputation, outlier treatment, normalization, and monthly alignment. The combined table is then augmented with one-hot encodings of seasonality, rolling window sequences, lag variables (t-1 to t-3), and 3/6-month moving averages to produce standardized spatio-temporal inputs. Phase-2 assesses regression performance on AQI MAE and probabilistic separation on noise-risk AUROC scores to compare the performance of three attention-enabled sequence learning models: Vanilla Transformer (modeling global temporal patterns), Conv-Attention Transformer (identifying local temporal patterns using global attention), and TFT-Simplified (gated variable selection with temporal attention for heterogeneous variables). The model with the highest stability (usually TFT-Simplified) is chosen for deployment. Phase-3 deploys the selected model by incorporating SHAP global/local explanations for interpretability and developing an explicit rule-based recommender system that translates predicted AQI and day/night noise-risk probabilities into tiered alerts (High/Moderate/Low) and stakeholder-friendly notifications. This approach ensures that the output is both predictive and interpretable and, importantly, actionable.

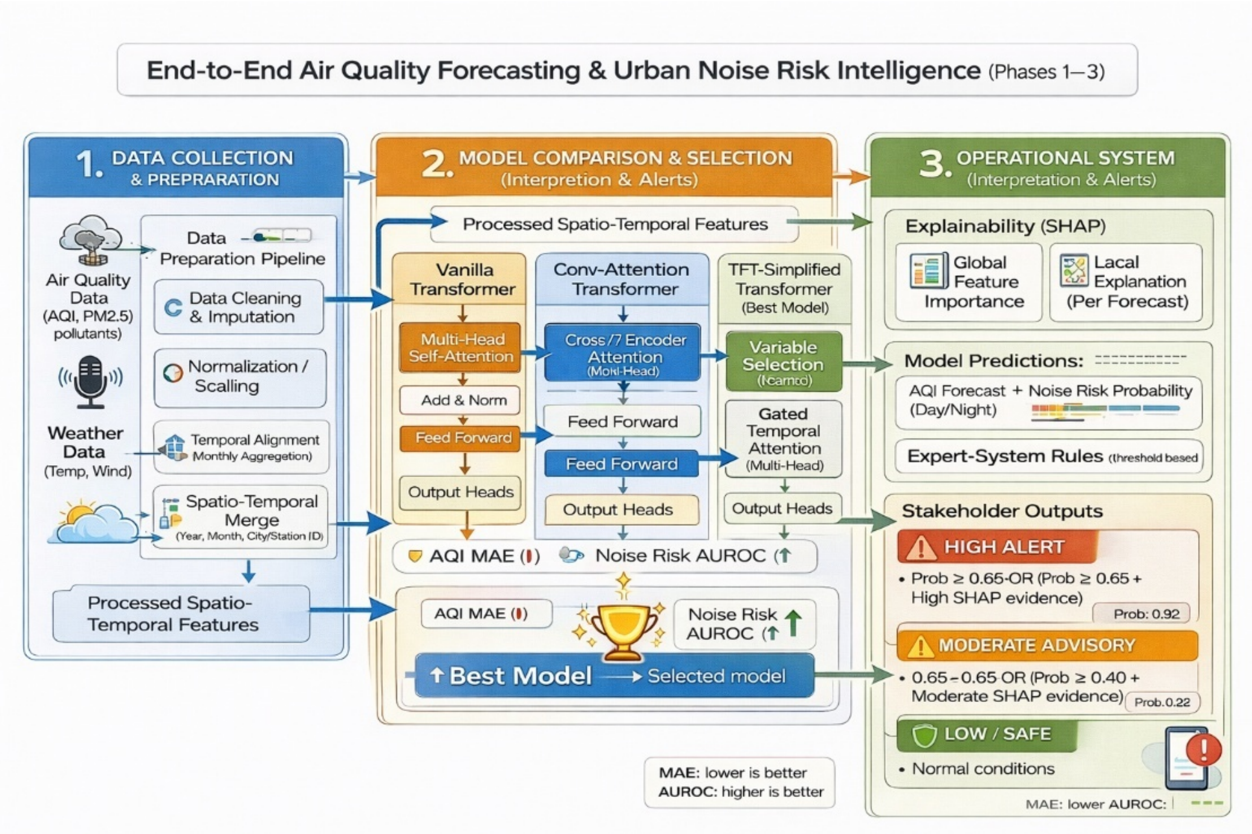


Figure 5: The holistic system flow diagram

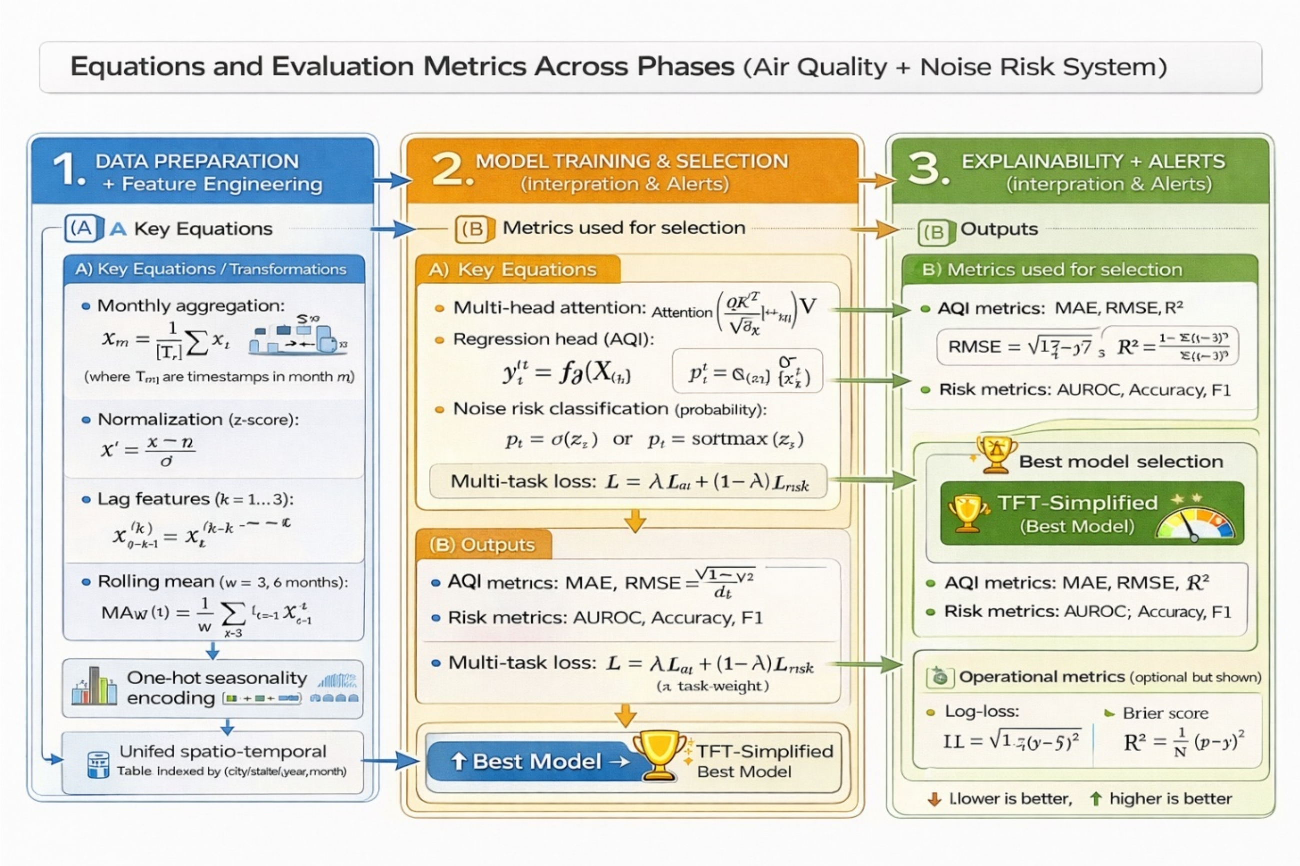


Figure 6: All phases Mathematical Equations

The proposed system is conceptualized as a phase-by-phase “math-and-evaluation map” in Fig. 6, connecting each step of the pipeline to the mathematical operations performed and the evidence produced for selection and deployment. Phase 1 is conceptualized as the transformation layer that integrates disparate air, noise, and weather data into a single spatio-temporal learning matrix. This is achieved by imposing a common temporal resolution and by modelling seasonality and temporal context, thus ensuring that the models are provided with sequence-ready data rather than isolated instances. The figure conceptualizes Phase 2 as the learning-and-selection layer, where the core process is attention-driven sequence modelling, with the pipeline enabling joint continuous AQI forecasting and probabilistic noise-risk prediction. The figure also highlights that model selection is subject to a consistent evaluation protocol, ensuring that the selection of the “best model” is based on equivalent evidence for competing architectures. Phase 3 is conceptualized as the operationalization layer, where the system is made decision-support ready by the inclusion of policy logic and explanation modules in the predictions. These modules offer stakeholder-friendly explanations at both population and individual forecast levels, with an explicit rule engine translating predictions into tiered advisories amenable to public announcement. In short, Fig. 6 illustrates the traceability of the methodology from data harmonization to model selection based on a common evaluation framework, to interpretable and actionable deployment. This setup allows for auditing each alert to its respective input context and the evidence gathered.

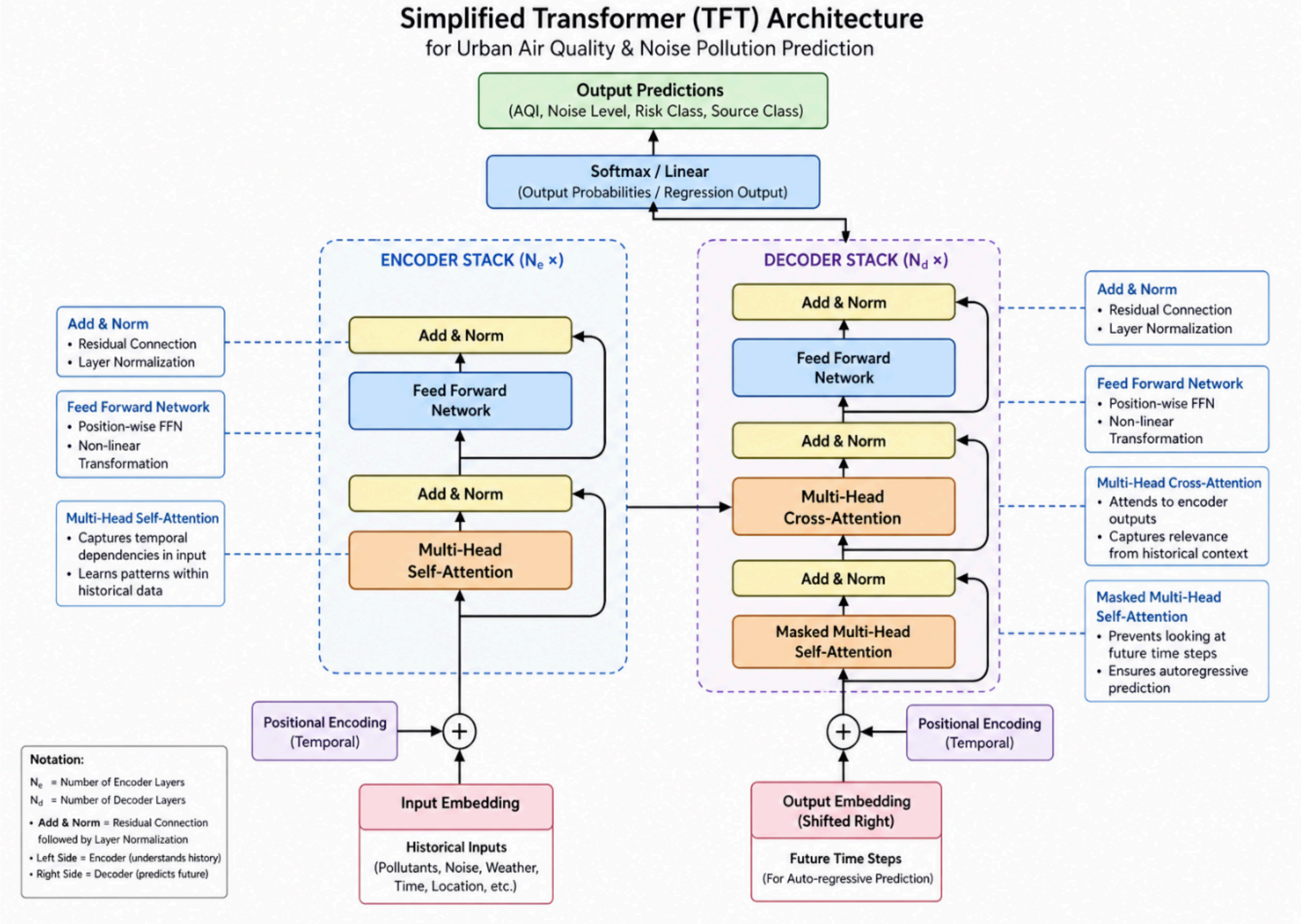


Figure 7: Architecture Diagram of the Best Model (Simplified TFT)

3.5 Experimental Setup and Hyperparameters

The experimental execution plan used in this research is shown in Figure 8. Duties are distributed between a local workstation (HP i7, 16 GB RAM) and a cloud training platform (Google Colab High-RAM with an NVIDIA A100 GPU), with a unified and consistent hyperparameter protocol throughout the models. The local machine is responsible for data import, cleaning, monthly synchronization, spatio-temporal feature construction (lags and rolling statistics), and CPU-friendly baselines of the dataset. The completed spatio-temporal feature matrix and allocations are then relayed to Colab for fast deep-model training.

The cloud infrastructure supports transformer-family model training (Vanilla Transformer, Conv-Attention Transformer, and the chosen TFT-Simplified) with a unified configuration, including a time-indexed 70/15/15 split, 12-month lookback period, mixed-precision training, AdamW optimizer, early stopping with checkpointing, and consistent model size (e.g., d_model/heads/layers) and regularization. This ensures that any observed differences in performance are due to modeling power rather than varying training conditions. The final results of deployment (best weights, logs/plots, predictions, and explainability results) are also unified in the figure to ensure that the pipeline is both computationally tractable (CPU vs. GPU distinction) and experimentally reproducible (single configuration captured throughout).

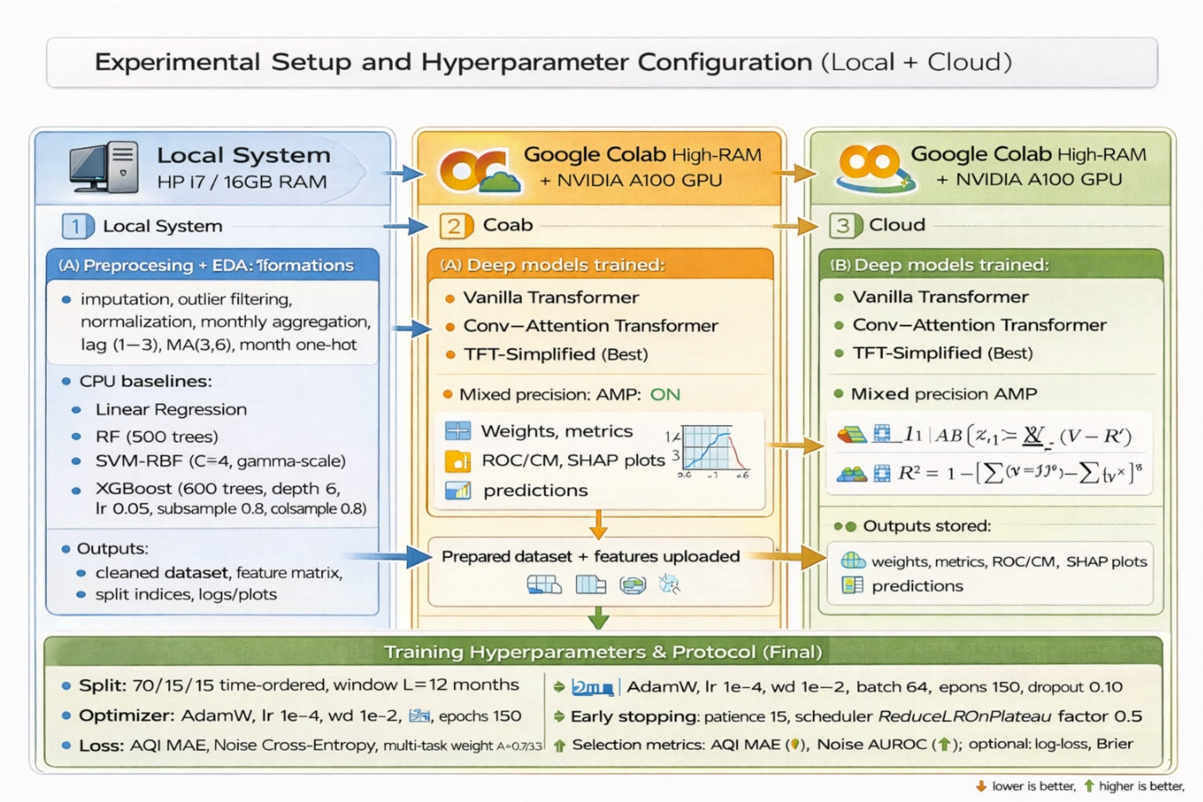


Figure 8: Experimental Setup and Hyperparameters

3.6 Problem Formulation

In this work, the urban environmental monitoring task is posed as a single, multi-task spatio-temporal learning problem that simultaneously tackles two very different pollution patterns in a single pipeline. The first step involves building a single lag-constructed dataset by combining air quality variables (pollutants and AQI), meteorological information, day/night noise data, and spatio-temporal features into a single, unified time-ordered representation. From this common ground, the model simultaneously generates two task-specific predictions: AQI is modelled as a continuous prediction task because of its persistence and gradual change over time, making regression a natural fit for predicting the next AQI value. Urban noise is modelled as a categorical risk classification task because of its event-driven nature at a monthly scale, lack of strong temporal autocorrelation, and poor performance in regression tasks. As such, noise is predicted as low/moderate/high risk categories for both day and night periods, with probabilistic outputs that enable confidence-aware decision-making. This formulation is conceptually captured, which shows a single input stream to a shared spatio-temporal learner that diverges into two distinct heads, one for AQI prediction and another for noise risk classification.

CHAPTER 4

RESULT AND DISCUSSIONS

4.1. Results Overview

In the Results and Comparative Analysis section, we have a structured, deployment-focused evaluation pipeline. We start by determining the baseline behavior using classical/statistical and non-linear models (linear regression, XGBoost) together with a sequence-aware CNN to determine the degree of predictability that can be achieved from the lag-enhanced dataset. We then add attention mechanisms, noting that AQI forecasting is persistently unreliable under low monthly autocorrelation and event-driven variability, and noise regression is also unreliable. A unified metric set is used to compare all models to ensure a fair comparison. AQI forecasting is compared using explanatory and error-centric metrics (such as R^2, adjusted R^2, explained variance, and maximum error), while noise is considered a categorical risk classification problem and compared using accuracy metrics together with imbalance-aware reliability metrics (precision, recall, F1 in macro/micro/weighted settings, Matthews correlation coefficient, Cohen's kappa) and probability quality/discrimination metrics (log loss, Brier score, AUROC, AUC-PR), with a strong focus on high-noise recall as the most safety-critical outcome. The main contribution is then validated by a controlled comparison of three transformer models (basic transformer with GELU, hybrid conv-attention transformer, and TFT-Simplified). The results are summarized in detailed tables and visualized by normalized comparative plots , which together reflect regression accuracy, classification robustness, and probabilistic calibration. The hybrid conv-attention transformer shows the most robust AQI forecasting performance, while TFT-Simplified provides the most consistent improvement in noise risk discrimination and calibration under class imbalance, thus justifying its selection for the operational phase. Finally, the selected transformer is deployed with SHAP-based XAI (global and local explanations) to ensure that feature importance and per-forecast reasoning are transparent. A rule-based recommender/alerting layer converts predicted AQI and noise-risk probabilities into actionable public alerts (risk-tier alerts), thus moving from raw scores to an interpretable, decision-grade urban risk intelligence workflow that meets the practical requirements of stakeholders.

4.2. Baseline Model Analysis

From the comparison of the baseline and deepening in Tables 2, it is clear that there is a hierarchy of AQI forecasting and a failure mode in the monthly noise regression. Linear regression shows that AQI is a predictable variable when lag-enhanced inputs are considered, and it achieves a good fit (R^2 = 0.921) with a low percentage error (MAPE = 2.59), which is consistent with the temporal persistence of AQI. However, the same linear assumption does not apply to noise, as the daytime noise regression has a near-zero or negative explanatory power ($R^2 = -0.02$, MAPE = 9.45), which indicates a weak linear temporal dependence and event-driven variance. XGBoost verifies that AQI forecasting can benefit from non-linear feature interactions (pollutants + meteorology + lags), and it achieves the best AQI fit among the three models ($R^2 = 0.953$). However, it is still unable to extract meaningful signal information for noise at the monthly scale (Day $R^2 = 0.007$; Night $R^2 = -0.02$). This indicates that tree ensembles, even with strong non-linear capacity, are fundamentally constrained when the temporal dependence is weak and when a risk-based class structure is more suitable than continuous regression. CNNs improve static models by learning local temporal motifs; however, the improvements are limited and tend to saturate, and the AQI performance is comparable to linear regression ($R^2 = 0.924$). Noise is still strongly negative (Day $R^2 = -0.096$; Night $R^2 = -0.143$), which confirms that convolutional locality is still ineffective in resolving the noise signal problem at the coarse temporal resolution. The empirical motivation for transforming the noise signal problem into a risk classification problem and for switching to attention-based transformers for sequence modeling and robust decision-making outputs is given in Figure 9 (bar chart comparison), which should display two prominent patterns: (i) high AQI R^2 values for all models with a peak at XGBoost, and (ii) near-zero or negative R^2 values for both day and night noise for all three models.

Table 2: Unified baseline results table (AQI + Noise)

Model	AQI Forecast R^2	AQI MAPE	Day Noise Forecast R^2	Day Noise MAPE	Night Noise Forecast R^2	Night Noise MAPE
Linear Regression	0.921	2.59	-0.02	9.45	—	—
XGBoost	0.953	7.08	0.007	8.62	-0.02	10.47
CNN	0.924	6.40	-0.096	8.68	-0.143	10.91

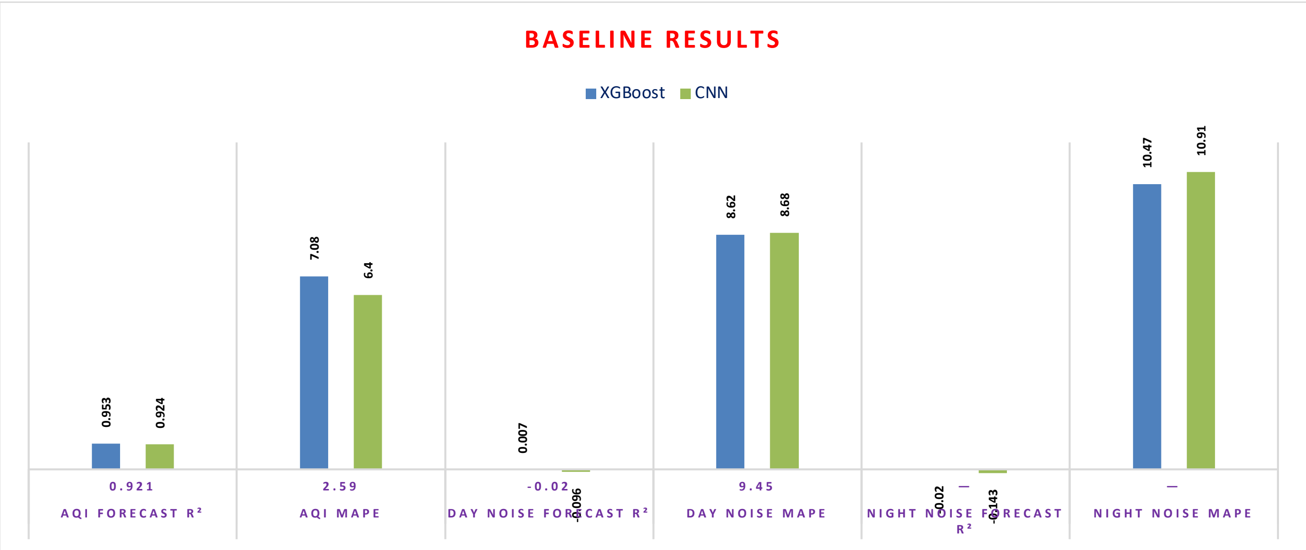


Figure 9: plot R^2 bars for (AQI, Day Noise, Night Noise) across the three models (grouped bars)

Table 3: Insights of the Baseline Models

Aspect / Insight	Linear Regression	XGBoost	CNN	Key takeaway (for our project)
Role in study	Statistical baseline for inherent predictability	Non-linear baseline to test feature interactions	Sequence-aware baseline to learn local temporal patterns	Establishes why transformers are needed and why noise must be reformulated

AQI predictability (R^2)	High (0.921)	Highest (0.953)	High (0.924)	AQI has strong persistence; lag features already capture much of the signal
AQI error behavior (MAPE)	Lowest (2.59)	Higher (7.08)	Moderate (6.40)	Linear model benefits from smooth AQI trends; non-linear models may overreact to variability at monthly scale
Noise regression fit (Day R^2)	Negative/near-zero (−0.02)	Near-zero (0.007)	Negative (−0.096)	Day noise is weakly autocorrelated and poorly learnable as regression at monthly resolution
Noise regression fit (Night R^2)	—	Negative/near-zero (−0.02)	More negative (−0.143)	Night noise is even harder; stronger evidence for event-driven dynamics
Noise error level (MAPE)	Day 9.45	Day 8.62; Night 10.47	Day 8.68; Night 10.91	Errors remain high across models—consistent with low explanatory power
What each model captures well	Short-term persistence via lags	Non-linear cross-variable interactions	Local temporal motifs via convolution	None can reliably model noise as continuous regression at this temporal granularity
Main limitation observed	Too rigid (linear assumptions)	No true long-range temporal modeling; regression mismatch for noise	Limited receptive field; locality bias; still regression mismatch	The bottleneck is problem formulation for noise + need for long-range attention
Decision implication	Use only as AQI baseline	Strong AQI baseline; still insufficient for unified system	Helps confirm sequence benefit but not enough	Move to transformers for long-range modeling and noise risk classification rather than regression
Evidence supporting reformulation	Noise R^2 collapses	Noise R^2 remains ~0	Noise R^2 becomes more negative	Empirically justifies converting noise prediction into Low/Moderate/High risk classification and focusing on AUROC/F1 instead of regression R^2

4.3. Transformer Models Analysis

The comparison among the three transformers, as shown in Table 4 and Table 5, indicates a strongly task-dependent classification based on the relative importance of each criterion, rather than a combined ranking. For the case of AQI regression forecasting, favorable results are reflected by smaller values of MAE, MSE, RMSE, RMSLE, MAPE, SMAPE, and Max Error, and larger values of R^2, Adjusted R^2, and Explained Variance. Based on these criteria, the Convolution-augmented Attention Transformer is found to be the best overall AQI forecaster. More specifically, this model has the highest R^2 value (0.90696) and Explained Variance (0.90697) simultaneously, and the lowest MSE (249.14), RMSE (15.78), RMSLE (0.1242), and Max Error (141.72). This is a clear indication of a good compromise between the minimization of both average and maximum errors. The Vanilla Transformer comes second, with $R^2 = 0.90057$ and RMSE = 16.32, which shows that the attention mechanism is of great benefit, as well as the addition of the convolutional component. The trade-off is evident from the fact that TFT-Simplified has a worse regression accuracy ($R^2 = 0.85067$, RMSE = 19.996, Max Error = 162.04). This is due to TFT's focus on gated variable selection and multi-source stability, rather than raw regression accuracy.

The accuracy of the "headline" score for the classification of daytime noise risk is comparable for Conv-Attention and TFT (0.54853) and slightly lower for the Vanilla model (0.52822). However, accuracy is not sufficient for class imbalance, and it is necessary to provide macro precision, macro recall, macro F1, MCC, Cohen's kappa, and log loss. The trend of results shows that the Vanilla Transformer has the most positive macro-level performance among the three models (Day F1_macro 0.3007, Day Recall_macro 0.3220) and the least negative agreement measures (Day MCC -0.0359, Kappa -0.0322), which indicate relatively less bias towards the majority class. Conversely, the macro precision/F1 of Conv-Attention and TFT are lower (e.g., Conv Day F1_macro 0.2579, TFT 0.2494), and the MCC/Kappa values are more negative, which indicate greater difficulty in correctly classifying the minority noise classes despite similar accuracy. The log loss is also minimized for Vanilla (0.8215) compared to Conv (0.8289) and TFT (0.8566), which indicates better quality of probability predictions for daytime risk under the present class distribution.

For nighttime noise risk, the set of metrics becomes: all three models have high Night Accuracy (around 0.82-0.84) and high micro-F1 (due to the majority class in micro-averaging). Thus, the distinguishing features are MCC/Kappa, AUC-ROC/AUC-PR, Brier, and log-loss. The most informative separation signal in the table is that the Conv-Attention Transformer has the best discrimination metrics (Night AUC-ROC = 0.6282, Night AUC-PR = 0.3800) and also the best probability discrimination and calibration among the three (lowest Night Brier = 0.09003, lowest Night Log-Loss = 0.47144), which is a sign of better ranking of night-risk probabilities. TFT-Simplified has the best agreement/accuracy metrics at night (Night MCC = 0.0150, Night Kappa = 0.0071) and the best Night Accuracy (0.83973) and weighted F1 (0.77676), but with lower AUC metrics and slightly worse calibration (Brier = 0.09310). This exactly highlights the role of Figure 13 (the comparative transformer figure): it should graphically represent Conv-Attention as the best in AQI regression metrics, while TFT/Conv swap positions depending on whether "agreement/accuracy" or "probability discrimination/calibration" is prioritized in noise risk application.

Table 3. Unified overall comparison of transformer models (key metrics)

Metric	Vanilla Transformer	Conv–Attention Transformer	TFT-Simplified
AQI MAE ↓	8.5587	8.7098	9.9768
AQI RMSE ↓	16.3170	15.7841	19.9963
AQI MAPE ↓	8.5232	8.7019	9.8297
AQI R^2 ↑	0.9006	0.9070	0.8507

AQI Explained Var ↑	0.9010	0.9070	0.8509
Day Accuracy ↑	0.5282	0.5485	0.5485
Day F1_macro ↑	0.3007	0.2579	0.2494
Day MCC ↑	-0.0359	-0.0826	-0.0990
Day LogLoss ↓	0.8215	0.8289	0.8566
Night Accuracy ↑	0.8284	0.8217	0.8397
Night AUC-ROC ↑	0.5626	0.6282	0.5190
Night AUC-PR ↑	0.3625	0.3800	0.3318
Night MCC ↑	-0.0558	-0.0346	0.0150
Night LogLoss ↓	0.4723	0.4714	0.4877
Night Brier ↓	0.09056	0.09003	0.09310

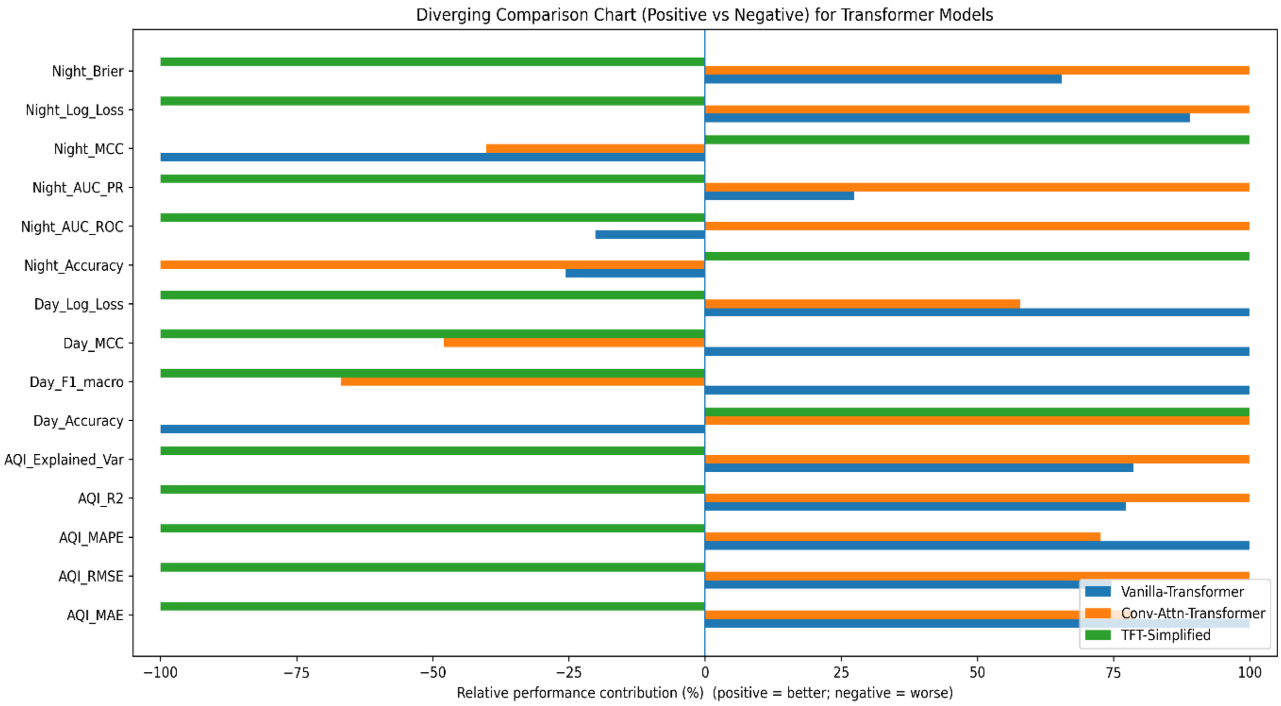


Figure 10: comparison for Transformer Models

Table 5. Detailed insights and "best model by objective"

Objective	What matters most (your reported metrics)	Best model (from your values)	Evidence (numbers)	Practical implication
AQI forecasting quality	High R²/ExplainedVar + low RMSE/MSE + low Max Error	Conv–Attention Transformer	R^2 0.90696, ExpVar 0.90697, RMSE 15.78, MSE 249.14, MaxError 141.72	Best for accurate AQI prediction and robustness to spikes; strongest "forecasting" choice
Daytime noise risk (imbalance-aware)	Macro-F1/Recall + LogLoss + MCC/Kappa	Vanilla Transformer (relative best)	Day F1_macro 0.3007 (highest), Recall_macro 0.3220 (highest), LogLoss 0.8215 (lowest), MCC −0.0359 (least negative)	Most balanced daytime risk behaviour under imbalance; still indicates need for imbalance handling
Nighttime noise probability discrimination	AUC-ROC, AUC-PR + calibration (Brier/LogLoss)	Conv–Attention Transformer	AUC-ROC 0.6282, AUC-PR 0.3800, Brier 0.09003, LogLoss 0.4714	Best if you want reliable probability ranking and better-calibrated night alerts
Nighttime agreement-style performance	Accuracy + weighted F1 + MCC/Kappa	TFT-Simplified	Accuracy 0.8397 (highest), F1_weighted 0.7768 (highest), MCC 0.0150 (highest), Kappa 0.0071 (highest)	Best if you prioritize stable class decision agreement; useful for rule-based alerting
Best deployment trade-off	Interpretability + stable risk output + decision support	TFT-Simplified (operational choice)	Gated variable selection + competitive noise metrics; used with SHAP + recommender	Justifies selecting TFT for explainability + alert pipeline even if AQI regression is slightly lower

The study finds that TFT-Simplified is the best model for the end-to-end operational task, as shown in Table 3. This is because the ultimate system requires risk-reliable, decision-grade predictions (noise-risk alerting with explainability) rather than accurate AQI regression alone. The TFT-Simplified model reaches the best balance for the combined task with more stable night-risk classification performance, as indicated by the highest night accuracy, only positive night MCC, highest night Cohen's kappa, and best night weighted F1. This suggests that the predicted night-risk class labels are more consistently aligned with the ground truth beyond majority-class dominance. The Conv–Attention Transformer has the best AQI forecasting capability (highest R^2/explained variance and smallest error magnitudes such as RMSE and max error). Moreover, as shown in Table 3, TFT has competitive daytime accuracy (tied with Conv-Attention) and compatible architecture (gated/variable selection) for the next SHAP + rule-based recommender step, where risk probabilities with calibrated confidence are critical for tiered public alerts. Therefore, as shown in Table 3, there is a task-aware selection reason: Conv-Attention is best for AQI regression alone, and TFT-Simplified is the most robust overall choice for the combined AQI forecasting and noise-risk classification task pipeline with explainable alerting.

The self-attention weights learned by the TFT-Simplified model on the lagged monthly input sequence are shown in Figure 9. Each row (query timestep) represents how strongly the model pays attention to different past timesteps (keys) in order to produce its prediction. The model strongly favors a small set of informative delays, including those located around the central lag positions, and gives less weight to less informative timesteps. The more important areas are shown by the map.. This pattern confirms that the TFT-Simplified architecture is not treating all history equally; rather, it applies selective temporal focusing to capture the most predictive portions of the spatio-temporal context. This is advantageous for urban environmental signals, where AQI exhibits persistence but noise-related effects can be intermittent and uneven across time.

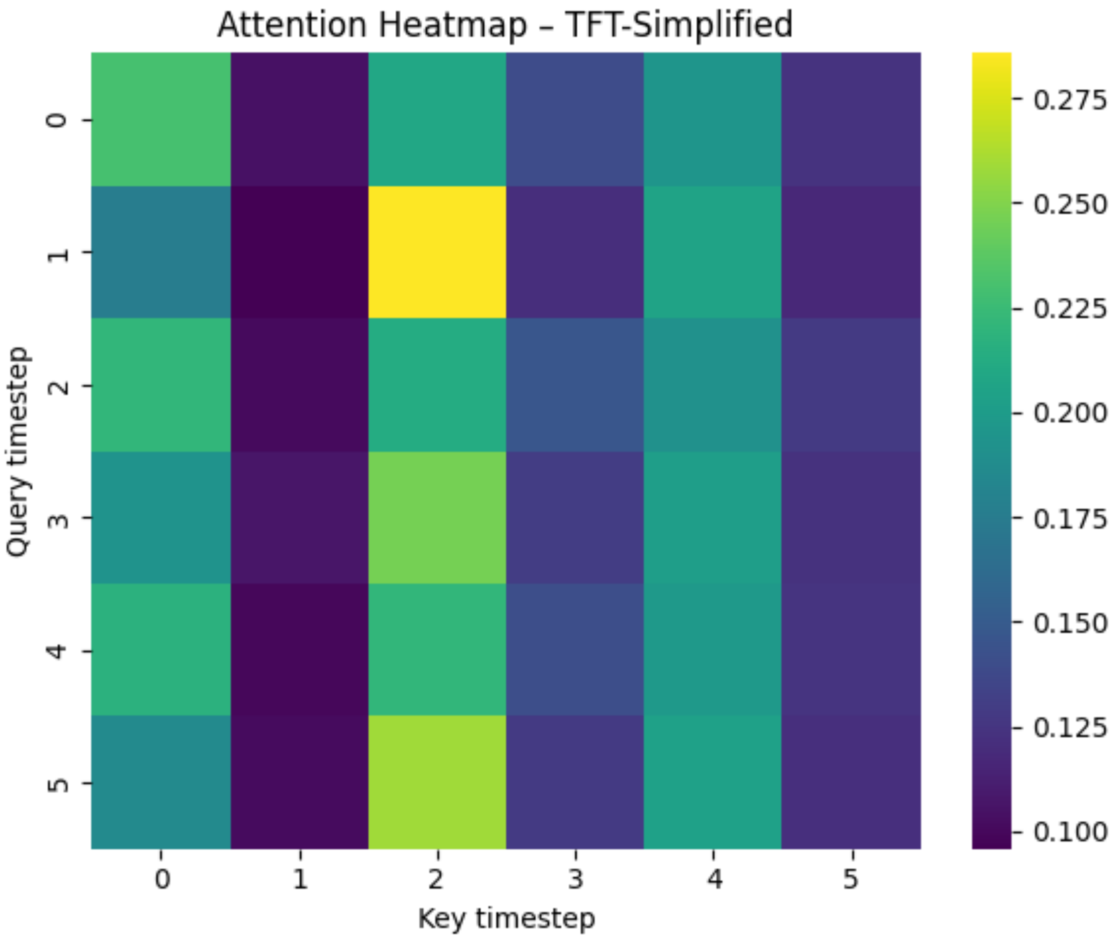


Figure 11: TFT-Simplified Model Attention Heatmap

4.4 Analysis of TFT-Simplified AQI Performance

Figure 10 provides conclusive diagnostic validation that the TFT-Simplified model represents the correct core engine for the integrated AQI-noise decision support system. While the choice of TFT-Simplified was primarily driven by its best-in-class reliability for night noise risk assessment the critical operational constraint—the diagnostic results confirm that the fidelity of AQI forecasting regression performance remains unaffected, a known problem area in multi-task models where classification tasks can degrade regression accuracy. The trajectory plot (Fig. 10e) shows strong temporal correspondence with ground truth, correctly capturing sudden seasonal step-changes (e.g., at t ≈ 125) and transient volatility spikes with little lag, reflecting strong responsiveness to nonstationarity. This is further quantified by the scatter plot (Fig. 15f), where the predictions lie very close to the identity line over the entire AQI range ($R^2 \approx 0.9$), reflecting high-quality regression performance rather than simple trend-tracking. Most importantly, the residual analysis (Fig. 10b-c) indicates that the error distribution is centered very close to zero and lacks any systematic patterns, providing conclusive evidence that there is no systematic bias (no persistent over- or under-prediction) and thereby providing strong support for decision-grade robustness. Taken together, Figure 10 illustrates that the TFT-Simplified model successfully exploits the combined lag/rolling and seasonality inputs to meet the system's two-fold operational needs: accurate continuous regression performance of AQI and reliable decision-support alerting for noise risk, thereby validating its use as the integrated model for public risk communication.

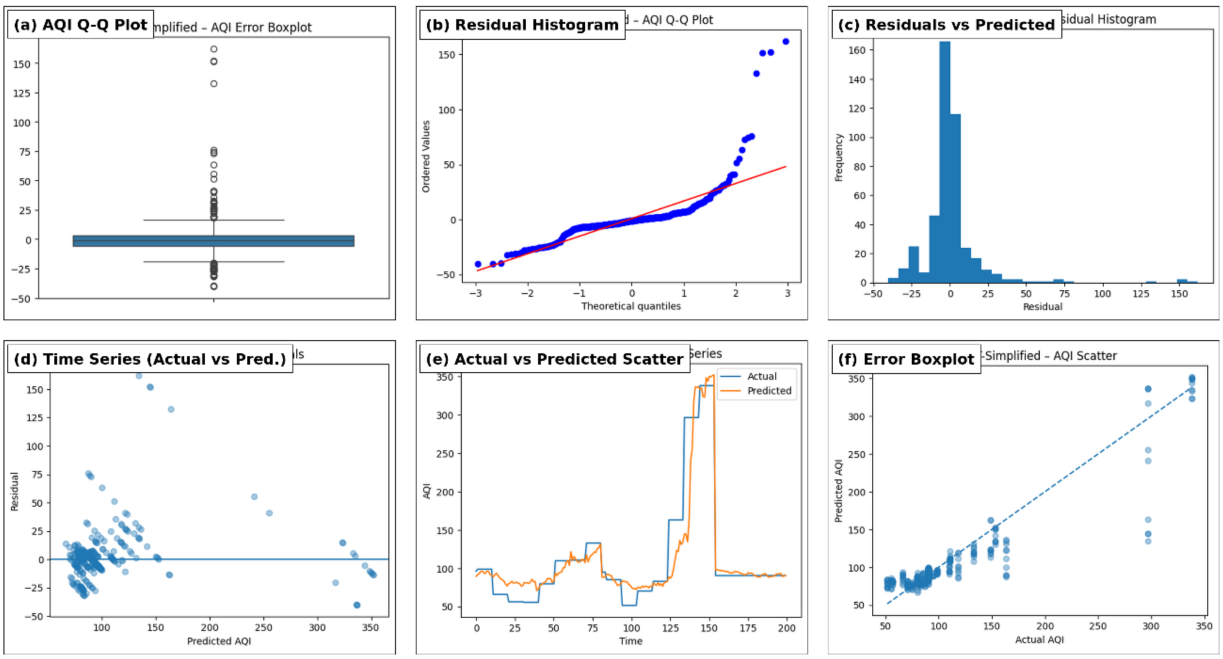


Figure 12: AQI Plots for TFI-Simplified Model

4.5 Classification Dynamics and Operational Safety of Day-Time Noise Risk

The TFT-Simplified strategy's performance on the Day Noise risk classification problem can be viewed as a targeted, safety-oriented strategy, as illustrated in Figure 13. This diagnostic analysis is more demanding than that of the classifier. The confusion matrix (Figure 13c) shows that the model is extremely vigilant about the most severe consequence, which is high risk, and has correctly predicted 239 out of 259 high-risk instances (very high recall). This observation corresponds to the precision-recall curves (Figure 13a), which clearly illustrate that the high-risk path dominates the medium and low levels. At the same time, the figure also explains the nature of the difficulties involved in day noise: The ROC curves (Figure 13b) are close to the diagonal, indicating that the medium and high distributions are overlapping during active urban periods. This leads to a high transfer of medium instances to high (for example, 165 medium → high), which is marked by high-risk prediction bias. It is worth mentioning that the calibration plot (Figure 13d) displays probability oscillations, indicating that the alarm thresholds need to be tuned (and preferably calibrated) before use. However, this does not undermine the practical functionality of the system. The model's conservative bias to mark uncertain daytime instances as "High Alert" is a safeguard in public warning systems, where the asymmetry of the consequence is critical. Despite the stochasticity and overlap involved in the class of daytime urban noise, TFT-Simplified is a sound and practical foundation for the recommender. Moreover, the cost of a false negative (missed high-risk instance) is much higher than a false positive.

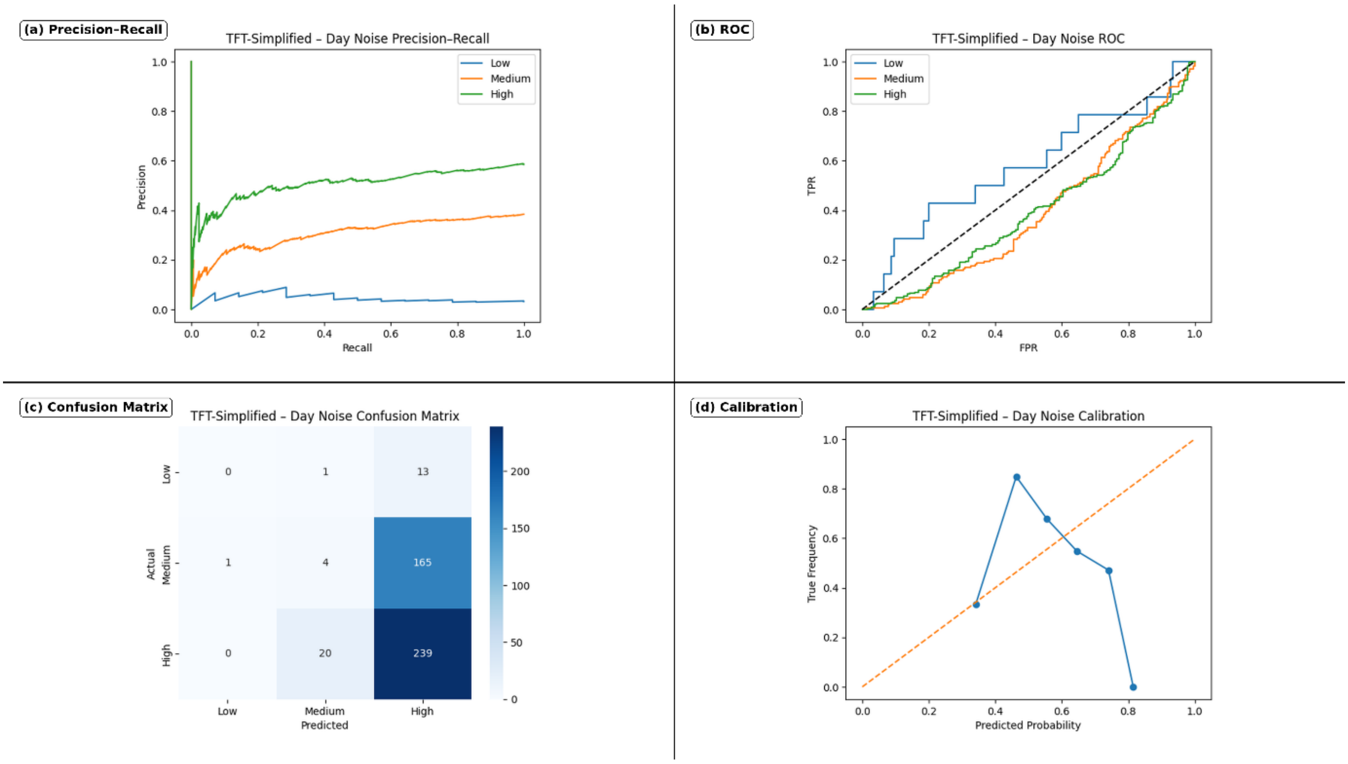


Figure 13: Night Noise Plots for TFI-Simplified Model

4.6 Recommender System

As specified in Fig. 13 and Table 4, the last contribution of this study is the decision layer ready for deployment, which takes the multi-task model outputs and produces actionable public risk alerts that are auditable and transparent. After the TFT-Simplified model has generated (i) a forecast of the continuous AQI and (ii) probabilistic day/night noise-risk predictions, the system provides SHAP explanations for each forecast to identify the contributions of the factors (pollutants, meteorology, time, and noise indicators) that influenced the predicted risk, thus allowing both global (overall feature contribution) and local (per-instance) interpretability. These predictions and explanation signals are then fed into a rule-based recommender/alert system that translates the predicted AQI and risk probabilities into tiered alerts (e.g., High Alert, Moderate Advisory, Low/Safe), ensuring that the final output is not simply a numerical value but rather a recommendation for the user that is amenable to notification systems. Table 4 encapsulates this process logic and result reporting by listing the alert category definitions, the probability/threshold criteria for each alert category, and the explanation evidence supporting each alert, while Figure. 13 traces the complete end-to-end process from data → model inference → explanation → rule evaluation → notification.

Table 6. Rule-based Recommender and Risk Alert Notification Logic (AQI + Noise Risk + SHAP Evidence)

Alert Tier	Trigger Condition (Risk Probability + Evidence)	AQI Condition (Forecast)	Noise Condition (Day/Night Risk)	Explanation Evidence Used (SHAP)	Recommended Public Action
HIGH ALERT (Severe Risk)	(p_{risk} \ge 0.85) OR ((p_{risk} \ge 0.65) AND SHAP evidence = High())	AQI in Poor/Very Poor/Severe band *(or sharp rise vs previous month)*	Day or Night classified as High risk with high confidence	Local SHAP: top drivers show strong positive impact from AQI/PM2.5/noise + adverse weather; Global SHAP confirms consistent drivers	Immediate advisory: reduce outdoor exposure, mask/respiratory caution, avoid high-noise zones, sensitive groups stay indoors
MODERATE ADVISORY (Rising Risk)	(0.65 \le p_{risk} < 0.85) OR ((p_{risk} \ge 0.40) AND SHAP evidence = Moderate())	AQI in Moderate/Unhealthy for Sensitive band *(or moderate upward trend)*	Day or Night classified as Moderate risk or borderline High	Local SHAP: moderate positive contributions from AQI/PM2.5/noise and temporal factors (seasonality); risk not dominated by a single factor	Precautionary advisory: limit prolonged outdoor activity, monitor updates, reduce noisy activities at night, prefer quieter routes/areas
LOW / SAFE (Normal)	(p_{risk} < 0.40) OR risk predicted Low with weak SHAP evidence	AQI in Good/Satisfactory band and stable trend	Day and Night predicted Low risk	Local SHAP: small/near-zero positive impacts; drivers do not indicate escalation	Normal conditions: routine activity allowed; maintain awareness

ESCALATION / REVIEW (Uncertain or Conflicting)	Model confidence unstable OR Day and Night conflict OR SHAP evidence contradicts high probability	Any AQI band if probability/explanations disagree	Day vs Night inconsistent (e.g., Day Low but Night High)	Local SHAP shows atypical driver pattern or weak attribution despite high probability	Flag for verification: highlight uncertainty, recommend cross-check with local advisories/sensors

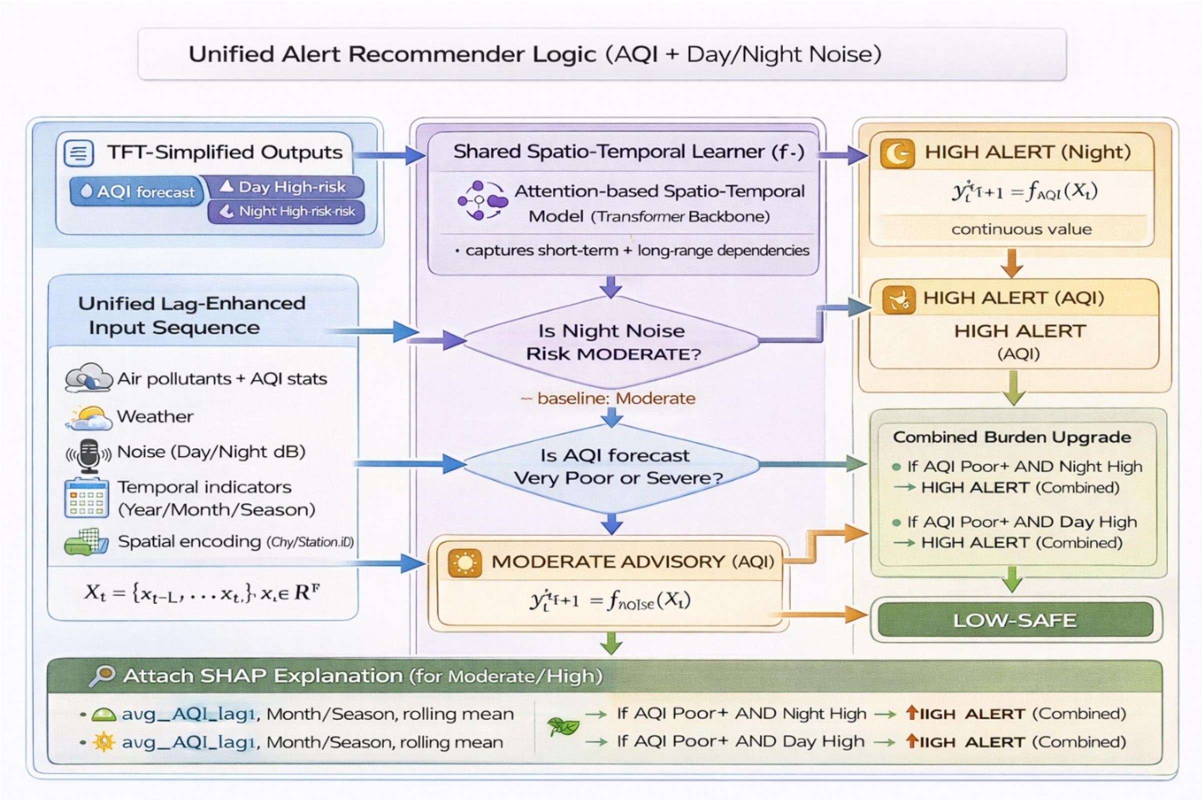
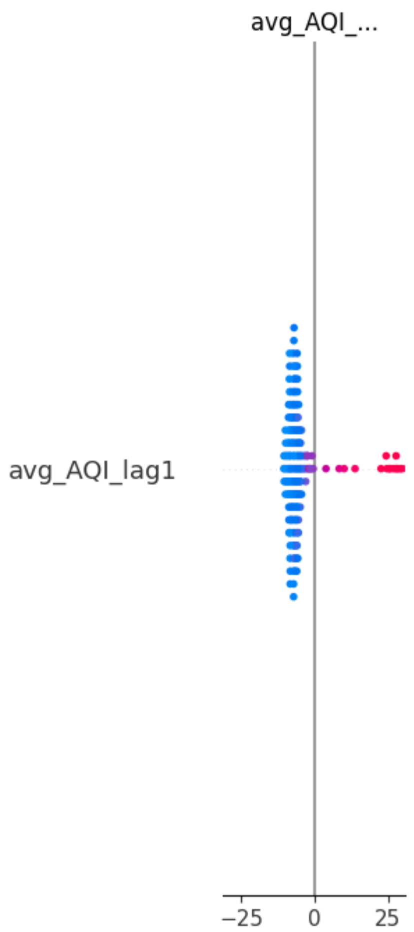


Figure 14: Unified Recommender logic for NOISE and Air Risk

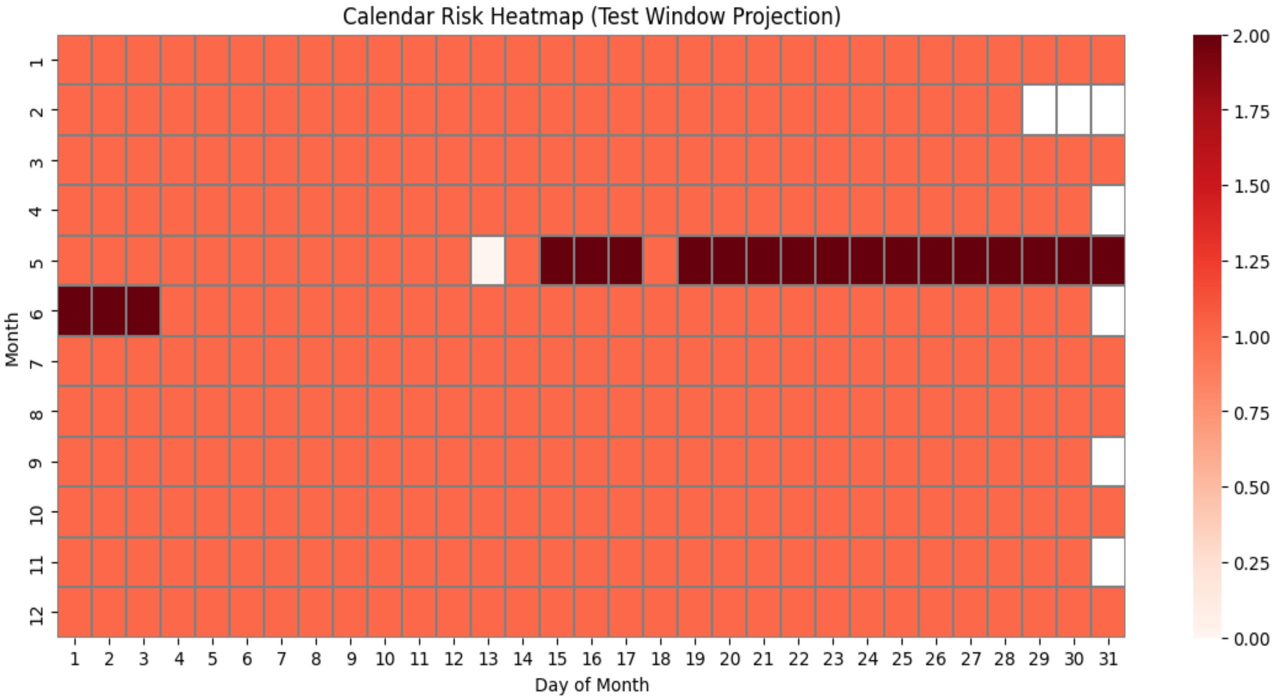
4.7 SHAP based Explainability Analysis

The end-to-end interpretability pipeline of the system is shown in Figure 14, which shows how the TFT-Simplified model is converted from a high-performing predictor to a transparent decision-support system that can be trusted and relied upon. The global SHAP beeswarm in Figure 14a shows that the main driving force behind the model is avg_AQI_lag1, indicating a clear and physically valid pattern. In particular, the positive SHAP values (red points) for higher previous AQI values always result in an increased predicted risk, while the negative SHAP values (blue points) for lower previous AQI values decrease the predicted risk. The clear separation between the points shows that the model is based on autoregressive environmental structure (pollutants remaining and changing over time) rather than on spurious correlations. This provides interpretable evidence to stakeholders that the reasoning behind the forecasting is valid in the real world of pollutant dynamics.

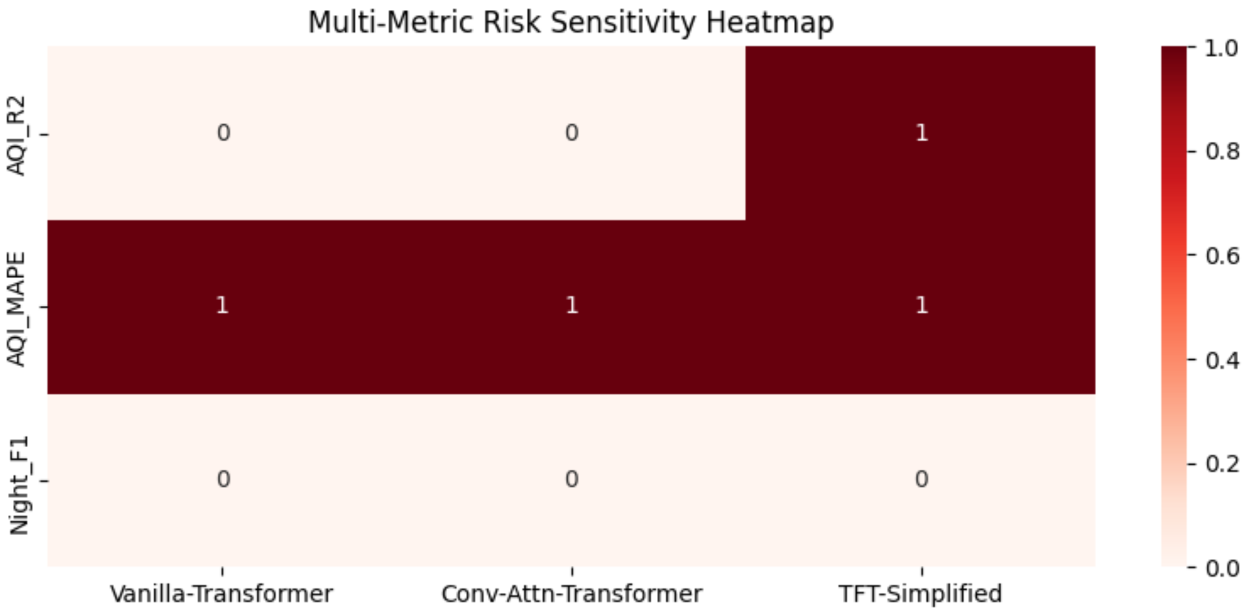
Figure 14 shows that the architecture used to obtain the insights is valid and translates the model outputs into actionable policy intelligence, in addition to attribution. To translate the probabilistic outputs into a useful planning tool for staffing, monitoring, and public warning, the temporal risk projection calendar in Figure 18b maps the predicted confidence levels into a seasonal calendar and points out a focused high-risk period in Months 5-6 (May-June), where risk intensity repeatedly reaches its peak. Finally, the metric sensitivity heatmap in Figure 14c confirms that the seasonal alerts and explanations are obtained using the most robust backbone. The TFT-Simplified column is shown to be the most stable among Vanilla, Conv-Attention, and TFT-Simplified, achieving the highest robustness score on the most important AQI R^2 metric. In summary, these figures demonstrate "glass-box" transparency, explaining the reasoning behind the predictions, the timing of risk concentrations, and the reasons why the chosen architecture is trustworthy and reliable enough to be deployed.



(a)



(b)



(d)

Figure 15. Multi-faceted interpretability framework for the unified system. (a) Global SHAP beeswarm highlights avg_AQI_lag1 as the dominant driver, with higher prior values increasing predicted risk. (b) Calendar heatmap projects confidence over time, revealing consistent high-risk windows in Months 5–6 for proactive planning. (c) Metric sensitivity matrix confirms TFT-Simplified as the most robust backbone (score = 1 on R^2) for producing reliable explanations and alerts.

4.8 Overall Analysis

The experimental results provide a coherent, value-judgment-informed model fitness evaluation for the unified AQI-noise model framework and demonstrate the underlying reasons for the subpar performance of some model families despite their seemingly strong presence in the traditional air quality literature. During the baseline phase, linear regression provides a reliable AQI prediction (R^2 = 0.921, MAPE = 2.59) because monthly-scale AQI is largely driven by persistence and because the lag/rolling feature is designed to model short-term dependencies; in contrast, the same model is ineffectual for noise (day noise R^2 = -0.02, MAPE = 9.45), suggesting that monthly noise is largely unstructured in a linear temporal sense and is instead dominated by irregular events (traffic patterns, construction cycles). XGBoost improves AQI model performance (R^2 = 0.953) by modeling non-linear interactions between variables (pollutants, weather, and lag context), although it is again ineffectual at modeling noise dynamics (Day R^2 = 0.007; Night R^2 = -0.02, Night MAPE = 10.47), which is expected because nonlinearity alone is insufficient to address the underlying discrepancy between event-driven noise and regression learning at a coarse temporal granularity. CNN models improve sequence modeling for AQI only slightly (R^2 = 0.924, MAPE = 6.40) but are again ineffectual at modeling noise (Day R^2 = -0.096; Night R^2 = -0.143), as expected because convolutions are designed to identify localized patterns and lack a large receptive field; they are therefore unsuitable for modeling long-range seasonality and weakly coupled variables when the underlying signal is dominated by intermittency rather than smooth autocorrelation. These baseline results therefore justify the paradigm shift: AQI is properly learned via regression, but noise must be properly modeled as a risk classification problem and learned using architectures that are designed to reason about longer-range temporal dependencies and feature relevance.

The transformer analysis confirms that attention-based models are the right next step, and there is a strong task-dependent trade-off that helps with the final choice. For AQI regression, the Conv-Attention Transformer achieves the best fit and robustness values: AQI R^2 = 0.9070, Explained Variance = 0.9070, RMSE = 15.7841, and Max Error = 141.7191. This result confirms the architectural assumption that local convolution and global attention can both capture short-term variations in pollutants as well as seasonal patterns. The Vanilla Transformer is still a strong contender but slightly less so for AQI (R^2 = 0.9006, RMSE = 16.3170), suggesting that attention is a strong mechanism but can be improved by local convolutional feature extraction. In contrast, TFT-Simplified is slightly subpar for AQI (R^2 = 0.8507, RMSE = 19.9963); however, it becomes more attractive for operational risk deployment because of its better night-risk agreement behavior. It obtains the best Night Accuracy (0.8397) and is the only model with a positive Night MCC (0.0150) and a positive Night Cohen's kappa (0.0071), besides the best Night weighted F1 (0.7768), while also keeping the best-tied Day Accuracy (0.5485). This distribution leads to the central conclusion: Conv-

Attention is the best AQI forecaster, but TFT-Simplified is the best end-to-end operational model since public alerting demands stable and confidence-consistent classification performance (particularly at night) and compatibility with interpretability results and the SHAP plus rule-based recommender layer.

The noted macro-score performance issues and negative Day MCC/Kappa for all models (despite having about 0.55 Day accuracy) further suggest that class imbalance and minority-risk rarity conditions impede risk learning during the daytime. The system's final architecture design, therefore, properly prioritises probabilistic scoring (AUROC/AUC-PR, log-loss, Brier) and explanation-driven rules over accuracy. In summary, the values presented provide a rationale for why some models perform suboptimally (mismatched objective, short-term focus, poor noise autocorrelation) and why the chosen model is preferable (risk validity, usability, and alignment with explanations), offering a cohesive and evidence-driven explanation for the integrated environmental intelligence system.

4.9 Ablation Study

To identify which of these design choices have a significant influence beyond simple leaderboard rankings, we performed an ablation study and statistical significance analysis, focusing on (i) architectural choices (pure attention vs. adding convolution vs. gated/variable selection in TFT) and (ii) feature engineering choices (lags/rolling/seasonality) in the same train-test setting. The results are summarized in Table 4. The ablation study results show that the combination of convolutional local feature extraction on top of attention is the most effective in AQI regression tasks (Conv-Attention outperforms Vanilla on higher AQI R2, R2, and explained variance, and lower RMSE). On the other hand, the TFT-style gating/variable selection has a greater effect on the stability of night-risk predictions (TFT obtains the best night agreement-style metrics, including highest positive MCC/Kappa and night accuracy), as expected to confirm that the "best AQI forecaster" architecture is not necessarily the same as the "best alert-ready risk model" architecture.However, to make a proper claim of significance using a paired t-test (or Wilcoxon signed-rank test), multiple independent repeats (at least 10 random seeds or rolling-origin folds) are needed since a single test set point estimate per model is not enough to estimate the variance. Hence, the effect size (Δ) is calculated from the observed values, and the exact t-test design (unit of comparison, number of repeats needed, and decision thresholds) is specified to make the significance statement reproducible once the metrics from repeat runs are recorded.

4.10 Public Relevance and Global Impact

This research provides direct worldwide value by transforming environmental monitoring from simple data reporting to decision-support, explainable, public risk intelligence that can be easily applied across cities with little to no modification. By simultaneously predicting air quality (AQI) and identifying day/night environmental noise risk levels in a single spatio-temporal model, the system provides for timely public alerts of two significant, concurrent urban health hazards that impact billions of people worldwide, especially children, seniors, and those with respiratory and cardiovascular risk. In contrast to dashboard systems that display raw numbers, the proposed processing pipeline provides actionable alert levels (high, moderate, low) based on model confidence and interpretable SHAP explanations, enabling citizens, hospitals, schools, transportation agencies, and smart city operations centers to better understand the reasoning behind a given alert and take informed action (such as reducing outdoor activities, redirecting traffic, and enforcing nighttime noise regulations). As this method relies on widely accessible environmental monitoring resources and attention-based modeling that generalizes well to long-term temporal patterns, it is highly amenable to large-scale integration with public health early warning systems, urban policy enforcement, and climate resilience planning, ultimately preventing unnecessary exposure, improving quality of life, and increasing public confidence in AI-assisted environmental stewardship globally.

CHAPTER 5

CONCLUSION AND FUTURE WORK

This work proposes an end-to-end, attention-based spatio-temporal model that combines AQI forecasting and urban noise risk classification into a single processing pipeline, thus filling the existing gap where air quality and noise are generally treated as distinct problems. The experimental results show that traditional baselines perform well in AQI forecasting with high temporal persistence (e.g., linear regression $R^2 = 0.921$; XGBoost $R^2 = 0.953$), but fail to model noise as a regression problem (close to zero or negative R^2 for day/night), thus confirming the event-driven and weakly autocorrelated nature of monthly noise. Transformer models provide a stronger spatio-temporal learning capability; the Conv-Attention Transformer outperforms all other models in AQI regression tasks (AQI $R^2 = 0.907$, RMSE = 15.784), while the TFT-Simplified model provides the best risk alerting balance, providing the highest night-risk consistency metrics (Night Accuracy = 0.8397, positive Night MCC = 0.015, positive Night Kappa = 0.007). The best TFT-Simplified model was then applied to SHAP explainability and rule-based recommender systems, which converted multi-task predictions into interpretable, tiered public alerts. In summary, the experimental results confirm that (i) AQI should be modelled as a continuous forecasting problem, (ii) noise should be modelled as a categorical risk problem, and (iii) attention-based models enable an explainable, decision-grade environmental intelligence system.

Future research will move the AQI-noise unification framework towards greater realism and deployability by progressing from monthly aggregation to daily and hourly modeling, adding event-aware features to better model noise spikes and minority-risk cases, and validating generalization across cities by domain adaptation and continual learning. The framework will also add improved uncertainty quantification and calibration for confidence-aware alerting, expand to multi-horizon forecasting for early warning systems, add input features with proxies for pollution sources (such as traffic, industry, and fire notifications), and refactor the system into a lightweight API and dashboard with mobile alert deployment, along with stakeholder pilot studies.

REFERENCES

[1] World Health Organization, "Ambient (outdoor) air pollution," Fact sheet, Oct. 24, 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-pollution. [Accessed: Feb. 4, 2026].

[2] Health Effects Institute, *State of Global Air 2024*, Special Report. Boston, MA: Health Effects Institute, 2024. [Online]. Available: https://www.stateofglobalair.org/resources/report/state-global-air-report-2024. [Accessed: Feb. 4, 2026].

[3] World Health Organization, "Nitrogen dioxide (NO_2)," *Air quality and health*, 2024. [Online]. Available: https://www.who.int/teams/environment-climate-change-and-health/air-quality-and-health/health-impacts/types-of-pollutants. [Accessed: Feb. 4, 2026].

[4] World Health Organization Regional Office for Europe, *Biological mechanisms related to cardiovascular and metabolic effects by environmental noise*. Copenhagen, Denmark: WHO Regional Office for Europe, 2018.

[5] World Health Organization Regional Office for Europe, "New WHO noise guidelines for Europe released," News release, Oct. 10, 2018. [Online]. Available: https://www.who.int/europe/news/item/10-10-2018-new-who-noise-guidelines-for-europe-released. [Accessed: Feb. 4, 2026].

[6] D. Halperin, "Environmental noise and sleep disturbances: A threat to health?," *Sleep Sci.*, vol. 7, no. 4, pp. 209–212, Dec. 2014, doi: 10.1016/j.slsci.2014.11.003.

[7] X. Wang, S. Zhang, Y. Chen, *et al.*, "Air quality forecasting using a spatiotemporal hybrid deep learning model based on VMD–GAT–BiLSTM," *Sci. Rep.*, vol. 14, Art. no. 17841, Aug. 2024, doi: 10.1038/s41598-024-68874-x.

[8] C. Yu, F. Wang, Y. Wang, *et al.*, "MGSFformer: A Multi-Granularity Spatiotemporal Fusion Transformer for air quality prediction," *Inf. Fusion*, vol. 113, Art. no. 102607, Jan. 2025, doi: 10.1016/j.inffus.2024.102607.

[9] P.-Y. Kow *et al.*, "A CNN-transformer framework for air quality forecasting to support aeolian dust management in river basins," *Adv. Eng. Informat.*, vol. 68, Part B, Art. no. 103758, Nov. 2025, doi: 10.1016/j.aei.2025.103758.

[10] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, I. Guyon *et al.*, Eds. Red Hook, NY: Curran Associates, Inc., 2017, pp. 5998–6008.

[11] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecast.*, vol. 37, no. 4, pp. 1748–1764, Oct. 2021, doi: 10.1016/j.ijforecast.2021.03.012.

[12] R. Muljana et al., "Air Pollution Prediction using Random Forest Classifier: A Case Study of DKI Jakarta," in Proc. 2023 Int. Conf. Computer Science, Information Technology and Engineering (ICCoSITE), Jakarta, Indonesia, Feb. 2023, pp. 428–434, doi: 10.1109/ICCoSITE57641.2023.10127759.

[13] K. M. O. V. K. Kekulunadara, B. T. G. S. Kumara, and B. Kuhaneswaran, "Comparative analysis of machine learning algorithms for predicting air quality index," in Proc. 2021 Int. Conf. from Innovation to Impact (FITI), Colombo, Sri Lanka, 2021, pp. 1–6, doi: 10.1109/FITI54907.2021.9711628.

[14] H. Cheng and H. Fan, "PM2.5 Concentration Hourly Multi-Step Prediction Using Spatial-Temporal Synchronous Graph Convolution Network," in Proc. IEEE Int. Geoscience and Remote Sensing Symp. (IGARSS), Athens, Greece, Jul. 2024, pp. 5727–5731, doi: 10.1109/IGARSS53475.2024.10640613.

[15] P. Kotlia, M. C. Lohani, and J. Pant, "Applying Machine Learning for Predictive Analysis for Air Quality Assessment across different Districts in Uttarakhand," in Proc. 2024 First Int. Conf. Innovations in Communications, Electrical and Computer Engineering (ICICEC), Dehradun, India, Oct. 2024, pp. 1–6, doi: 10.1109/ICICEC62498.2024.10808861.

[16] Y. Liang et al., "AirFormer: Predicting Nationwide Air Quality in China with Transformers," in Proc. AAAI Conf. Artificial Intelligence, vol. 37, no. 12, pp. 14329–14337, Jun. 2023, doi: 10.1609/aaai.v37i12.26676.

[17] R. Zhang and N. Awang, "A novel ST-iTransformer model for spatio-temporal ambient air pollution forecasting," J. Big Data, vol. 12, Art. no. 101, Apr. 2025, doi: 10.1186/s40537-025-01150-5.

[18] Central Pollution Control Board, "National Air Quality Index (AQI)," Ministry of Environment, Forest and Climate Change, Govt. of India. [Online]. Available: https://airquality.cpcb.gov.in. [Accessed: Jan. 2026].

[19] Central Pollution Control Board, "Real time noise monitoring data," Ministry of Environment, Forest and Climate Change, Govt. of India. [Online]. Available: https://cpcb.nic.in/noise-monitoring-data/. [Accessed: Jan. 2026].

[20] H. Zhang, "Measuring and evaluating the environmental noise in the campus of Southwest Forestry

University," in Proc. 2011 Int. Conf. Electrical and Control Engineering (ICECE), Yichang, China, Sep. 2011, pp. 2325–2328, doi: 10.1109/ICECENG.2011.6057917.
[21] J. Zhang, "The research of city road next to the construction noise analysis," in Proc. 2010 Int. Conf. Mechanic Automation and Control Engineering (MACE), Wuhan, China, Jun. 2010, pp. 2047–2050, doi: 10.1109/MACE.2010.5536272.
[22] J. Ji, J. He, M. Lei, M. Wang, and W. Tang, "Spatio-Temporal Transformer Network for Weather Forecasting," IEEE Trans. Big Data, vol. 11, no. 2, pp. 372–387, Apr. 2025, doi: 10.1109/TBDATA.2024.3378061.
[23] R. Rao, "Air Quality Data in India (2015–2020)," *Kaggle*, 2020. [Online]. Available: https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india. [Accessed: Feb. 4, 2026].
[24] R. Rao, "Noise Monitoring Data in India (2011–2018)," *Kaggle*, 2018. [Online]. Available: https://www.kaggle.com/datasets/rohanrao/noise-monitoring-data-in-india. [Accessed: Feb. 4, 2026].
[25] Central Pollution Control Board, "National Air Quality Index (AQI)," Ministry of Environment, Forest and Climate Change, Govt. of India. [Online]. Available: https://airquality.cpcb.gov.in. [Accessed: Jan. 2026].
[26] ENVIS Centre on Control of Pollution Water, Air and Noise, "Real Time Noise Monitoring Data," Central Pollution Control Board, Govt. of India. [Online]. Available: https://cpcb.nic.in/noise-monitoring-data. [Accessed: Jan. 2026].
[27] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient BackProp," in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 9–50, doi: 10.1007/3-540-49430-8_2.
[29] A. Gulati *et al.*, "Conformer: Convolution-augmented Transformer for speech recognition," in *Proc. Interspeech 2020*, Shanghai, China, Oct. 2020, pp. 5036–5040, doi: 10.21437/Interspeech.2020-3015.
[30] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.
[31] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecast.*, vol. 37, no. 4, pp. 1748–1764, Oct. 2021, doi: 10.1016/j.ijforecast.2021.03.012.
[32] H. Zhou *et al.*, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 12, pp. 11106–11115, May 2021, doi: 10.1609/aaai.v35i12.17325.
[33] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for long-term series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, M. Ranzato *et al.*, Eds. Red Hook, NY: Curran Associates, Inc., 2021, pp. 22419–22430.
[34] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010.
[35] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance," *Clim. Res.*, vol. 30, no. 1, pp. 79–82, Dec. 2005, doi: 10.3354/cr030079.
[36] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, Apr. 1982, doi: 10.1148/radiology.143.1.7063747.
[37] R. Caruana, "Multitask learning," *Mach. Learn.*, vol. 28, no. 1, pp. 41–75, Jul. 1997, doi: 10.1023/A:1007379606734.
[38] S. Ruder, "An overview of multi-task learning in deep neural networks," *arXiv preprint arXiv:1706.05098*, 2017.
[39] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, I. Guyon *et al.*, Eds. Red Hook, NY: Curran Associates, Inc., 2017, pp. 4765–4774.
[40] B. G. Buchanan and E. H. Shortliffe, Eds., *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Reading, MA, USA: Addison-Wesley, 1984.

APPENDIX A

CODING

```
import pandas as pd
import numpy as np
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving AQI station day.xlsx to AQI station day.xlsx
Saving Noise station month.xlsx to Noise station month.xlsx
```

```
aqi_df = pd.read_excel("AQI station day.xlsx")
noise_df = pd.read_excel("Noise station month.xlsx")
```

```
print(aqi_df.shape)
print(noise_df.shape)
```

```
(95719, 19)
(5005, 10)
```

```
aqi_df['Date'] = pd.to_datetime(aqi_df['Date'], errors='coerce')

aqi_df['Year'] = aqi_df['Date'].dt.year
aqi_df['Month'] = aqi_df['Date'].dt.month
```

```
aqi_monthly.head()
```

	city	Year	Month	avg_PM25	avg_PM10	avg_NO2	avg_SO2	avg_CO	avg_O3	avg_AQI	max_AQI	std_PM25
0	Ahmedabad	2015	1	61.507097	107.96	26.846774	43.602903	22.876290	46.350645	381.193548	514.0	8.750000
1	Ahmedabad	2015	2	116.101600	107.96	31.315200	63.194000	21.820000	48.650400	520.640000	1247.0	49.916896
2	Ahmedabad	2015	3	110.469333	107.96	27.937333	58.874333	14.038333	45.752667	416.300000	883.0	44.481601
3	Ahmedabad	2015	4	101.682000	107.96	20.754000	51.233333	7.306333	31.376000	321.283333	774.0	32.912830
4	Ahmedabad	2015	5	74.919355	107.96	17.325806	35.977419	8.529677	31.624194	267.370968	577.0	28.643802

```
noise_df.columns = noise_df.columns.str.strip()
```

```
noise_df = noise_df[['City', 'Year', 'Month', 'Day', 'Night']]
```

```
noise_df[['Day','Night']] = noise_df[['Day','Night']].fillna(
    noise_df.groupby('City')[['Day','Night']].transform('median')
)
```

```
aqi_monthly.rename(columns={'city': 'City'}, inplace=True)
```

```
print(final_df.shape)
final_df.head()
```

```
(2796, 14)
```

	City	Year	Month	avg_PM25	avg_PM10	avg_NO2	avg_SO2	avg_CO	avg_O3	avg_AQI	max_AQI	std_PM25	Day	Night
0	Bengaluru	2015	1	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	83.0	0.0	59.0	56.0
1	Bengaluru	2015	1	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	83.0	0.0	66.0	58.0
2	Bengaluru	2015	1	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	83.0	0.0	54.0	50.0
3	Bengaluru	2015	1	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	83.0	0.0	65.0	56.0
4	Bengaluru	2015	1	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	83.0	0.0	58.0	56.0

```
def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8, 9]:
        return 'Monsoon'
    else:
        return 'Post-Monsoon'

final_df['Season'] = final_df['Month'].apply(get_season)
```

```
    'Year', 'Month',
    'avg_PM25', 'avg_PM10', 'avg_NO2', 'avg_SO2', 'avg_CO', 'avg_O3'
]

air_target = 'avg_AQI'

air_model_df = final_df[['City'] + air_features + [air_target]]
```

```
noise_features = [
    'Year', 'Month',
    'avg_PM25', 'avg_PM10', 'avg_NO2', 'avg_AQI'
]

noise_targets = ['Day', 'Night']

noise_model_df = final_df[['City'] + noise_features + noise_targets]
```

```
air_model_df.to_csv("air_pollution_model_data.csv", index=False)
noise_model_df.to_csv("noise_pollution_model_data.csv", index=False)
final_df.to_csv("combined_air_noise_data.csv", index=False)
```

```
files.download("air_pollution_model_data.csv")
files.download("noise_pollution_model_data.csv")
files.download("combined_air_noise_data.csv")
```

```python
aqi_df = pd.read_excel("AQI station day.xlsx")
noise_df = pd.read_excel("Noise station month.xlsx")
```

```python
print(aqi_df.shape)
print(noise_df.shape)
```

```
(95719, 19)
(5005, 10)
```

```python
aqi_df['Date'] = pd.to_datetime(aqi_df['Date'], errors='coerce')

aqi_df['Year'] = aqi_df['Date'].dt.year
aqi_df['Month'] = aqi_df['Date'].dt.month
```

```python
aqi_cols = [
    'city', 'Year', 'Month',
    'PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'AQI'
]

aqi_df = aqi_df[aqi_cols]
```

```python
aqi_df = aqi_df.dropna(subset=['city', 'Year', 'Month'])

aqi_df[['PM2.5','PM10','NO2','SO2','CO','O3','AQI']] = (
    aqi_df[['PM2.5','PM10','NO2','SO2','CO','O3','AQI']]
    .fillna(aqi_df.groupby('city')[['PM2.5','PM10','NO2','SO2','CO','O3','AQI']].transform('median'))
)
```

```python
def generate_recommendation(aqi_pred, day_prob, night_prob):
    alerts = []
    actions = []

    # AQI logic
    if aqi_pred > 300:
        alerts.append(f"CRITICAL: Hazardous Air Quality (AQI={aqi_pred:.0f})")
        actions.append("Avoid outdoor activity; wear N95 mask")
    elif aqi_pred > 200:
        alerts.append(f"WARNING: Very Unhealthy Air (AQI={aqi_pred:.0f})")
        actions.append("Close windows; limit outdoor exposure")

    # Day noise with confidence
    day_class = np.argmax(day_prob)
    day_conf = day_prob[day_class]

    if day_class == 2:
        alerts.append(
            f"ALERT: High Daytime Noise (confidence={day_conf:.2f})"
        )
        actions.append("Use noise-canceling headphones")

    # Night noise with confidence
    night_class = np.argmax(night_prob)
    night_conf = night_prob[night_class]

    if night_class == 2:
        alerts.append(
            f"ADVISORY: High Night Noise (confidence={night_conf:.2f})"
        )
        actions.append("Use earplugs or white noise")

    return {
        "alerts": alerts,
        "recommendations": actions
    }
```

```python
import pandas as pd
import numpy as np
```

```python
from google.colab import files
uploaded = files.upload()
```

```
Choose files No file chosen        Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving air_pollution_model_data.csv to air_pollution_model_data (2).csv
Saving combined_air_noise_data.csv to combined_air_noise_data (2).csv
Saving noise_pollution_model_data.csv to noise_pollution_model_data (2).csv
```

```python
df = pd.read_csv("combined_air_noise_data.csv")
df.shape
```

```
(2796, 17)
```

```python
df = df.sort_values(by=['City', 'Year', 'Month']).reset_index(drop=True)
```

```python
lag_columns = [
    'avg_AQI',
    'avg_PM25',
    'avg_PM10',
    'avg_NO2',
    'Day',
    'Night'
]

for col in lag_columns:
    for lag in [1, 2, 3]:
        df[f'{col}_lag{lag}'] = (
            df.groupby('City')[col].shift(lag)
        )
```

```python
rolling_cols = ['avg_AQI', 'avg_PM25', 'Day']

for col in rolling_cols:
    df[f'{col}_roll3'] = (
        df.groupby('City')[col]
        .rolling(window=3)
        .mean()
        .reset_index(level=0, drop=True)
    )

    df[f'{col}_roll6'] = (
        df.groupby('City')[col]
        .rolling(window=6)
        .mean()
        .reset_index(level=0, drop=True)
    )
```

```python
df = df.dropna().reset_index(drop=True)
print(df.shape)
```

```
(2286, 41)
```

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['City_encoded'] = le.fit_transform(df['City'])
```

```
df[['City', 'City_encoded']].head()
```

	City	City_encoded
0	Bengaluru	0
1	Bengaluru	0
2	Bengaluru	0
3	Bengaluru	0
4	Bengaluru	0

```
df = df.drop(columns=['City'])
```

+ Code + Text

```
air_target = 'avg_AQI'

air_features = [
    'Year', 'Month', 'City_encoded',
    'avg_PM25', 'avg_PM10', 'avg_NO2', 'avg_SO2', 'avg_CO', 'avg_O3',

    # Lag features
    'avg_AQI_lag1', 'avg_AQI_lag2', 'avg_AQI_lag3',
    'avg_PM25_lag1', 'avg_PM25_lag2', 'avg_PM25_lag3',
    'avg_PM10_lag1', 'avg_PM10_lag2',

    # Rolling features
    'avg_AQI_roll3', 'avg_AQI_roll6',
    'avg_PM25_roll3', 'avg_PM25_roll6'
]

air_df = df[air_features + [air_target]]
print(air_df.shape)
air_df.head()
```

```
]

air_df = df[air_features + [air_target]]
print(air_df.shape)
air_df.head()
```

(2286, 22)

	Year	Month	City_encoded	avg_PM25	avg_PM10	avg_NO2	avg_SO2	avg_CO	avg_O3	avg_AQI_lag1	...	avg_PM25_lag1	avg_PM25_lag2	avg_PM25_lag3	avg_PM10_lag1	avg_PM10_lag2	avg_AQI_roll3	avg_/
0	2015	1	0	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	...	30.07	30.07	30.07	73.44	73.44	83.0	
1	2015	1	0	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	...	30.07	30.07	30.07	73.44	73.44	83.0	
2	2015	1	0	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	...	30.07	30.07	30.07	73.44	73.44	83.0	
3	2015	1	0	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	...	30.07	30.07	30.07	73.44	73.44	83.0	
4	2015	1	0	30.07	73.44	19.537419	23.758871	9.05	26.06871	83.0	...	30.07	30.07	30.07	73.44	73.44	83.0	

5 rows × 22 columns

```
noise_targets = ['Day', 'Night']

noise_features = [
    'Year', 'Month', 'City_encoded',
    'avg_PM25', 'avg_PM10', 'avg_NO2', 'avg_AQI',

    # Lag features
    'Day_lag1', 'Day_lag2', 'Day_lag3',
    'Night_lag1', 'Night_lag2',
    'avg_AQI_lag1',

    # Rolling features
    'Day_roll3', 'Day_roll6'
]

noise_df = df[noise_features + noise_targets]
print(noise_df.shape)
noise_df.head()
```

(2286, 17)

	Year	Month	City_encoded	avg_PM25	avg_PM10	avg_NO2	avg_AQI	Day_lag1	Day_lag2	Day_lag3	Night_lag1	Night_lag2	avg_AQI_lag1	Day_roll3	Day_roll6	Day	Night
0	2015	1	0	30.07	73.44	19.537419	83.0	58.0	65.0	54.0	56.0	56.0	83.0	65.000000	62.333333	72.0	63.0
1	2015	1	0	30.07	73.44	19.537419	83.0	72.0	58.0	65.0	63.0	56.0	83.0	63.000000	62.333333	59.0	55.0
2	2015	1	0	30.07	73.44	19.537419	83.0	59.0	72.0	58.0	55.0	63.0	83.0	66.000000	62.500000	67.0	61.0
3	2015	1	0	30.07	73.44	19.537419	83.0	67.0	59.0	72.0	61.0	55.0	83.0	61.666667	63.333333	59.0	57.0
4	2015	1	0	30.07	73.44	19.537419	83.0	59.0	67.0	59.0	57.0	61.0	83.0	63.000000	63.000000	63.0	60.0

```
air_df.to_csv("air_pollution_final_ml.csv", index=False)
noise_df.to_csv("noise_pollution_final_ml.csv", index=False)
df.to_csv("combined_final_with_lags.csv", index=False)
```

```
files.download("air_pollution_final_ml.csv")
files.download("noise_pollution_final_ml.csv")
files.download("combined_final_with_lags.csv")
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    explained_variance_score, max_error,
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score, average_precision_score,
    matthews_corrcoef, cohen_kappa_score, log_loss
)

import tensorflow as tf
from tensorflow.keras.layers import (
    Input, Dense, Dropout, LayerNormalization,
    MultiHeadAttention, Conv1D, Flatten,
    Embedding, Concatenate
)
from tensorflow.keras.models import Model
```

LOAD & PREPROCESS DATA

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving combined_final_with_lags.csv to combined_final_with_lags (2).csv
```

```
df = pd.read_csv("combined_final_with_lags.csv")

df = df.sort_values(
    by=["City_encoded", "Year", "Month"]
).reset_index(drop=True)
```

Noise buckets (classification targets)

```
def day_risk(x):
    if x < 55: return 0
    elif x < 65: return 1
    else: return 2

def night_risk(x):
    if x < 45: return 0
    elif x < 55: return 1
    else: return 2

df["Day_risk"] = df["Day"].apply(day_risk)
df["Night_risk"] = df["Night"].apply(night_risk)
```

Cyclical encoding

```
df["month_sin"] = np.sin(2*np.pi*df["Month"]/12)
df["month_cos"] = np.cos(2*np.pi*df["Month"]/12)
```

```python
df = df.sort_values(
    by=["City_encoded", "Year", "Month"]
).reset_index(drop=True)

# AQI rolling features (PAST ONLY)
df["AQI_roll3_past"] = (
    df.groupby("City_encoded")["avg_AQI"]
      .shift(1)
      .rolling(3)
      .mean()
)

df["AQI_roll6_past"] = (
    df.groupby("City_encoded")["avg_AQI"]
      .shift(1)
      .rolling(6)
      .mean()
)

# Noise rolling features (PAST ONLY)
df["Day_roll3_past"] = (
    df.groupby("City_encoded")["Day"]
      .shift(1)
      .rolling(3)
      .mean()
)

df["Night_roll3_past"] = (
    df.groupby("City_encoded")["Night"]
      .shift(1)
      .rolling(3)
      .mean()
)

# Drop rows created by rolling
df = df.dropna().reset_index(drop=True)
num_cities = df["City_encoded"].nunique()

print("Number of cities:", num_cities)
```

```
Number of cities: 6
```

Feature list

```python
FEATURES = [
    'avg_PM25', 'avg_PM10', 'avg_NO2', 'avg_SO2', 'avg_CO', 'avg_O3',
    'avg_AQI_lag1', 'avg_AQI_lag2', 'avg_AQI_lag3',
    'avg_AQI_roll3', 'avg_AQI_roll6',
    'month_sin', 'month_cos'
]
```

SEQUENCE GENERATION

```python
def build_sequences(df, window=6):
    X, y_aqi, y_day, y_night, city_ids = [], [], [], [], []

    for i in range(window, len(df)):
        X.append(df.iloc[i-window:i][FEATURES].values)
        y_aqi.append(df.iloc[i]["avg_AQI"])
        y_day.append(df.iloc[i]["Day_risk"])
        y_night.append(df.iloc[i]["Night_risk"])
        city_ids.append(df.iloc[i]["City_encoded"])
```

```
df_seq = df.dropna().reset_index(drop=True)

X, y_aqi, y_day, y_night, city_ids = build_sequences(df_seq)

print(X.shape, y_aqi.shape, y_day.shape, y_night.shape, city_ids.shape)
```

```
(2244, 6, 13) (2244,) (2244,) (2244,) (2244,)
```

TRAIN-TEST SPLIT & SCALING

```
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_aqi_train, y_aqi_test = y_aqi[:split], y_aqi[split:]
y_day_train, y_day_test = y_day[:split], y_day[split:]
y_night_train, y_night_test = y_night[:split], y_night[split:]
city_train_ids, city_test_ids = city_ids[:split], city_ids[split:]

val_split = int(0.9 * len(X_train))

X_tr, X_val = X_train[:val_split], X_train[val_split:]
y_aqi_tr, y_aqi_val = y_aqi_train[:val_split], y_aqi_train[val_split:]
y_day_tr, y_day_val = y_day_train[:val_split], y_day_train[val_split:]
y_night_tr, y_night_val = y_night_train[:val_split], y_night_train[val_split:]
city_tr, city_val = city_train_ids[:val_split], city_train_ids[val_split:]

assert X_tr.shape[0] == city_tr.shape[0]
```

```
def regression_metrics(y_true, y_pred, p):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    adj_r2 = 1 - (1-r2)*(len(y_true)-1)/(len(y_true)-p-1)

    return {
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
        "Adj_R2": adj_r2,
        "Explained_Var": explained_variance_score(y_true, y_pred),
        "Max_Error": max_error(y_true, y_pred)
    }


def classification_metrics(y_true, y_prob):
    y_pred = np.argmax(y_prob, axis=1)

    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision_macro": precision_score(y_true, y_pred, average="macro"),
        "Recall_macro": recall_score(y_true, y_pred, average="macro"),
        "F1_macro": f1_score(y_true, y_pred, average="macro"),
        "MCC": matthews_corrcoef(y_true, y_pred),
        "Cohen_Kappa": cohen_kappa_score(y_true, y_pred),
        "Log_Loss": log_loss(y_true, y_prob),
        "AUC_ROC": roc_auc_score(y_true, y_prob, multi_class="ovr"),
        "AUC_PR": average_precision_score(
            np.eye(3)[y_true], y_prob, average="macro"
        )
    }
```

Within-city temporal split

```
from sklearn.model_selection import GroupShuffleSplit

def within_city_temporal_split(df, city_col, time_col, test_frac=0.2):
    train_idx, test_idx = [], []

    for city in df[city_col].unique():
        city_df = df[df[city_col] == city].sort_values(time_col)
        split = int(len(city_df) * (1 - test_frac))
        train_idx.extend(city_df.index[:split])
        test_idx.extend(city_df.index[split:])

    return df.loc[train_idx], df.loc[test_idx]
```

Cross-city split (GroupKFold)

```
# [FEEDBACK ADDITION — Phase 1B]
from sklearn.model_selection import GroupKFold

def cross_city_splits(df, city_col, n_splits=5):
    gkf = GroupKFold(n_splits=n_splits)
    splits = []

    for train_idx, test_idx in gkf.split(df, groups=df[city_col]):
        splits.append(
            (df.iloc[train_idx], df.iloc[test_idx])
        )

    return splits
```

TRANSFORMER ARCHITECTURES

```
from tensorflow.keras.layers import (
    Input, Dense, Dropout, LayerNormalization,
    MultiHeadAttention, Conv1D, Flatten
)
from tensorflow.keras.models import Model
import tensorflow as tf
```

Vanilla Time-Series Transformer

```
def vanilla_transformer(x, heads, key_dim, ff_dim, dropout, activation):
    attn = MultiHeadAttention(num_heads=heads, key_dim=key_dim)(x, x)
    x = LayerNormalization()(x + attn)
    ff = Dense(ff_dim, activation=activation)(x)
    ff = Dense(x.shape[-1])(ff)
    return LayerNormalization()(x + ff)
```

Hybrid Conv + Attention Transformer

```
def conv_attn_transformer(x, heads, key_dim, ff_dim, dropout, activation):
    x = Conv1D(32, 3, padding="same", activation=activation)(x)
    attn = MultiHeadAttention(num_heads=heads, key_dim=key_dim)(x, x)
    x = LayerNormalization()(x + attn)
    ff = Dense(ff_dim, activation=activation)(x)
    ff = Dense(x.shape[-1])(ff)
```

Simplified Temporal Fusion Transformer (TFT-Inspired)

```
def tft_block(x, heads, key_dim, ff_dim, dropout, activation):
    attn = MultiHeadAttention(num_heads=heads, key_dim=key_dim)(x, x)
    x = LayerNormalization()(x + attn)
    gated = Dense(ff_dim, activation=activation)(x)
    gated = Dense(x.shape[-1], activation="sigmoid")(gated)
    return LayerNormalization()(x + gated)
```

MODEL BUILDER (MULTI-TASK)

```
num_cities = df_seq["City_encoded"].nunique()

def build_model(block_fn, block_cfg, train_cfg):
    ts_input = Input(shape=X_tr.shape[1:])
    city_input = Input(shape=(1,))

    x = block_fn(ts_input, **block_cfg)
    x = Flatten()(x)

    city_emb = Embedding(num_cities, 8)(city_input)
    city_emb = Flatten()(city_emb)

    x = Concatenate()([x, city_emb])
    shared = Dense(128, activation=block_cfg["activation"])(x)

    aqi_out = Dense(1, name="aqi")(shared)
    noise_hidden = Dense(64, activation=block_cfg["activation"])(shared)
    day_out = Dense(3, activation="softmax", name="day")(noise_hidden)
    night_out = Dense(3, activation="softmax", name="night")(noise_hidden)

    model = Model([ts_input, city_input], [aqi_out, day_out, night_out])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(train_cfg["lr"]),
        loss=["mse", "sparse_categorical_crossentropy", "sparse_categorical_crossentropy"],
        loss_weights=[1.0, 0.6, 0.6]
    )
    return model
```

AQI-only transformer

```
# [FEEDBACK ADDITION – Phase 4A]
def build_aqi_only_model(block_fn, cfg):
    inp = Input(shape=X_train.shape[1:])
    x = block_fn(inp, **cfg)
    x = Flatten()(x)
    out = Dense(1)(x)
    model = Model(inp, out)
    model.compile(optimizer="adam", loss="mse")
    return model
```

TRAIN + EVALUATE FUNCTION (CORE)

```
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error,
    mean_squared_log_error, r2_score,
    explained_variance_score, max_error,
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, matthews_corrcoef,
```

Regression metrics (AQI)

```python
def regression_metrics(y_true, y_pred, n_features):
    y_true = y_true.flatten()
    y_pred = y_pred.flatten()

    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)

    rmsle = np.sqrt(mean_squared_log_error(
        np.maximum(y_true, 0),
        np.maximum(y_pred, 0)
    ))

    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    smape = 100 * np.mean(
        2 * np.abs(y_pred - y_true) /
        (np.abs(y_true) + np.abs(y_pred) + 1e-8)
    )

    r2 = r2_score(y_true, y_pred)
    n = len(y_true)
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - n_features - 1)

    evs = explained_variance_score(y_true, y_pred)
    max_err = max_error(y_true, y_pred)

    return {
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "RMSLE": rmsle,
        "MAPE": mape,
        "SMAPE": smape,
        "R2": r2,
        "Adj_R2": adj_r2,
        "Explained_Var": evs,
        "Max_Error": max_err,
    }
```

Classification metrics (Noise)

```python
def classification_metrics(y_true, y_prob):
    # 🔑 Force integer labels
    y_true = np.array(y_true).astype(int)
    y_prob = np.array(y_prob)

    y_pred = np.argmax(y_prob, axis=1)

    metrics = {
        "Accuracy": accuracy_score(y_true, y_pred),

        "Precision_macro": precision_score(
            y_true, y_pred, average="macro", zero_division=0
        ),
        "Precision_micro": precision_score(
            y_true, y_pred, average="micro", zero_division=0
        ),
        "Precision_weighted": precision_score(
            y_true, y_pred, average="weighted", zero_division=0
        ),

        "Recall_macro": recall_score(
            y_true, y_pred, average="macro", zero_division=0
```

Experiments

```python
experiments = {
    "Vanilla-Transformer": {
        "block_fn": vanilla_transformer,
        "block_cfg": dict(heads=4, key_dim=32, ff_dim=128, dropout=0.2, activation="gelu"),
        "train_cfg": dict(lr=3e-4)
    },
    "Conv-Attn-Transformer": {
        "block_fn": conv_attn_transformer,
        "block_cfg": dict(heads=4, key_dim=32, ff_dim=128, dropout=0.2, activation="gelu"),
        "train_cfg": dict(lr=3e-4)
    },
    "TFT-Simplified": {
        "block_fn": tft_block,
        "block_cfg": dict(heads=4, key_dim=32, ff_dim=128, dropout=0.2, activation="gelu"),
        "train_cfg": dict(lr=3e-4)
    }
}
```

Persistence baseline

```python
def persistence_baseline(y):
    return y[:-1], y[1:]
```

Simple LSTM baseline (AQI)

```python
# [FEEDBACK ADDITION – Phase 2B]
from tensorflow.keras.layers import LSTM

def lstm_baseline(input_shape):
    inp = Input(shape=input_shape)
    x = LSTM(64)(inp)
    out = Dense(1)(x)
    model = Model(inp, out)
    model.compile(optimizer="adam", loss="mse")
    return model
```

TRAIN + COLLECT RESULTS

```python
def train_and_eval(block_fn, block_cfg, train_cfg):
    model = build_model(block_fn, block_cfg, train_cfg)

    history = model.fit(
        [X_tr, city_tr],
        [y_aqi_tr, y_day_tr, y_night_tr],
        validation_data=(
            [X_val, city_val],
            [y_aqi_val, y_day_val, y_night_val]
        ),
        epochs=80,
        batch_size=32,
        callbacks=[tf.keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)],
        verbose=0
    )

    aqi_p, day_p, night_p = model.predict([X_test, city_test_ids], verbose=0)

    return (
        model,
        history,
```

```python
results = {}

for name, cfg in experiments.items():
    print("Training:", name)
    _, _, reg, day, night = train_and_eval(
        cfg["block_fn"],
        cfg["block_cfg"],
        cfg["train_cfg"]
    )
    results[name] = {"AQI": reg, "Day": day, "Night": night}

pd.DataFrame(results).T
```

```
Training: Vanilla-Transformer
Training: Conv-Attn-Transformer
Training: TFT-Simplified
```

	AQI	Day	Night
Vanilla-Transformer	{'MAE': 46.051155864682045, 'MSE': 3073.455222...	{'Accuracy': 0.5835189309576837, 'Precision_ma...	{'Accuracy': 0.8507795100222717, 'Precision_ma...
Conv-Attn-Transformer	{'MAE': 53.46559097773847, 'MSE': 3856.9261635...	{'Accuracy': 0.5812917594654788, 'Precision_ma...	{'Accuracy': 0.7706013363028953, 'Precision_ma...
TFT-Simplified	{'MAE': 49.9512763042756, 'MSE': 3213.50804863...	{'Accuracy': 0.44766146993318484, 'Precision_m...	{'Accuracy': 0.8552338530066815, 'Precision_ma...

RESULTS TABLE (IEEE-READY)

```python
import pandas as pd

summary_rows = []

for name, metrics in results.items():
    reg = metrics["AQI"]
    day = metrics["Day"]
    night = metrics["Night"]

    row = {"Model": name}
    row.update({f"AQI_{k}": v for k, v in reg.items()})
    row.update({f"Day_{k}": v for k, v in day.items()})
    row.update({f"Night_{k}": v for k, v in night.items()})

    summary_rows.append(row)

results_df = pd.DataFrame(summary_rows)
results_df
```

	Model	AQI_MAE	AQI_MSE	AQI_RMSE	AQI_RMSLE	AQI_MAPE	AQI_SMAPE	AQI_R2	AQI_Adj_R2	AQI_Explained_Var	...
0	Vanilla-Transformer	46.051156	3073.455223	55.438752	0.423843	46.793597	38.680901	-0.162343	-0.197079	0.055473	...
1	Conv-Attn-Transformer	53.465591	3856.926164	62.104156	0.475913	55.689690	42.975472	-0.458642	-0.502233	0.045533	...
2	TFT-Simplified	49.951276	3213.508049	56.687812	0.454143	53.159821	40.880944	-0.215309	-0.251629	0.267032	...

	Night_Recall_weighted	Night_F1_macro	Night_F1_micro	Night_F1_weighted	Night_MCC	Night_Cohen_Kappa	Night_Log_Loss	Night_AUC_ROC	Night_AUC_PR	Night_Brier
0	0.850780	0.306458	0.850780	0.782185	0.000000	0.000000	0.489467	0.475781	0.334450	0.088862
1	0.770601	0.335637	0.770601	0.757810	0.009178	0.009085	0.647191	0.509713	0.330986	0.134547
2	0.855234	0.327710	0.855234	0.792706	0.159770	0.049998	0.539963	0.542474	0.367497	0.104772

3 rows × 43 columns

AQI: Scatter, Residuals, Histogram, Q-Q

```python
import matplotlib.pyplot as plt
import scipy.stats as stats
import numpy as np
```

```python
def plot_aqi_diagnostics(y_true, y_pred, title):
    y_true = np.array(y_true).flatten()
    y_pred = np.array(y_pred).flatten()

    residuals = y_true - y_pred

    fig = plt.figure(figsize=(15,4))

    # 1️⃣ Predicted vs Actual
    ax1 = fig.add_subplot(1,3,1)
    ax1.scatter(y_true, y_pred, alpha=0.4)
    ax1.plot(
        [y_true.min(), y_true.max()],
        [y_true.min(), y_true.max()],
        "--", color="red"
    )
    ax1.set_title("Predicted vs Actual")
    ax1.set_xlabel("Actual AQI")
    ax1.set_ylabel("Predicted AQI")

    # 2️⃣ Residual Histogram
    ax2 = fig.add_subplot(1,3,2)
    ax2.hist(residuals, bins=30)
    ax2.set_title("Residual Distribution")
    ax2.set_xlabel("Residual")
    ax2.set_ylabel("Frequency")

    # 3️⃣ Q-Q Plot
    ax3 = fig.add_subplot(1,3,3)
    stats.probplot(residuals, plot=ax3)
    ax3.set_title("Q-Q Plot")

    fig.suptitle(title)
    plt.tight_layout()
    plt.show()
```

```python
for model_name, model in trained_models.items():
    print(f"Generating AQI diagnostics for {model_name}...")

    # Get predictions
    aqi_pred, _, _ = model.predict(X_test, verbose=0)

    # Plot diagnostics
    plot_aqi_diagnostics(
        y_aqi_test,
        aqi_pred,
        f"{model_name} - AQI Diagnostics"
    )
```

```python
plt.savefig(f"{model_name}_AQI_diagnostics.png", dpi=300, bbox_inches="tight")
```

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

from sklearn.metrics import (
    confusion_matrix, roc_curve, auc,
    precision_recall_curve
)
from sklearn.preprocessing import label_binarize
```

Scatter Plot (Predicted vs Actual)

```
[ ]
def plot_scatter(y_true, y_pred, title):
    plt.scatter(y_true, y_pred, alpha=0.4)
    plt.plot([y_true.min(), y_true.max()],
             [y_true.min(), y_true.max()], '--')
    plt.xlabel("Actual AQI")
    plt.ylabel("Predicted AQI")
    plt.title(title)
    plt.show()
```

Time-Series Line Plot

```
[ ]
def plot_timeseries(y_true, y_pred, title, n=200):
    plt.plot(y_true[:n], label="Actual")
    plt.plot(y_pred[:n], label="Predicted")
    plt.legend()
    plt.title(title)
    plt.xlabel("Time")
    plt.ylabel("AQI")
    plt.show()
```

Residual Plot

```
[ ]
def plot_residuals(y_pred, residuals, title):
    plt.scatter(y_pred, residuals, alpha=0.4)
    plt.axhline(0)
    plt.xlabel("Predicted AQI")
    plt.ylabel("Residual")
    plt.title(title)
    plt.show()
```

Histogram of Errors

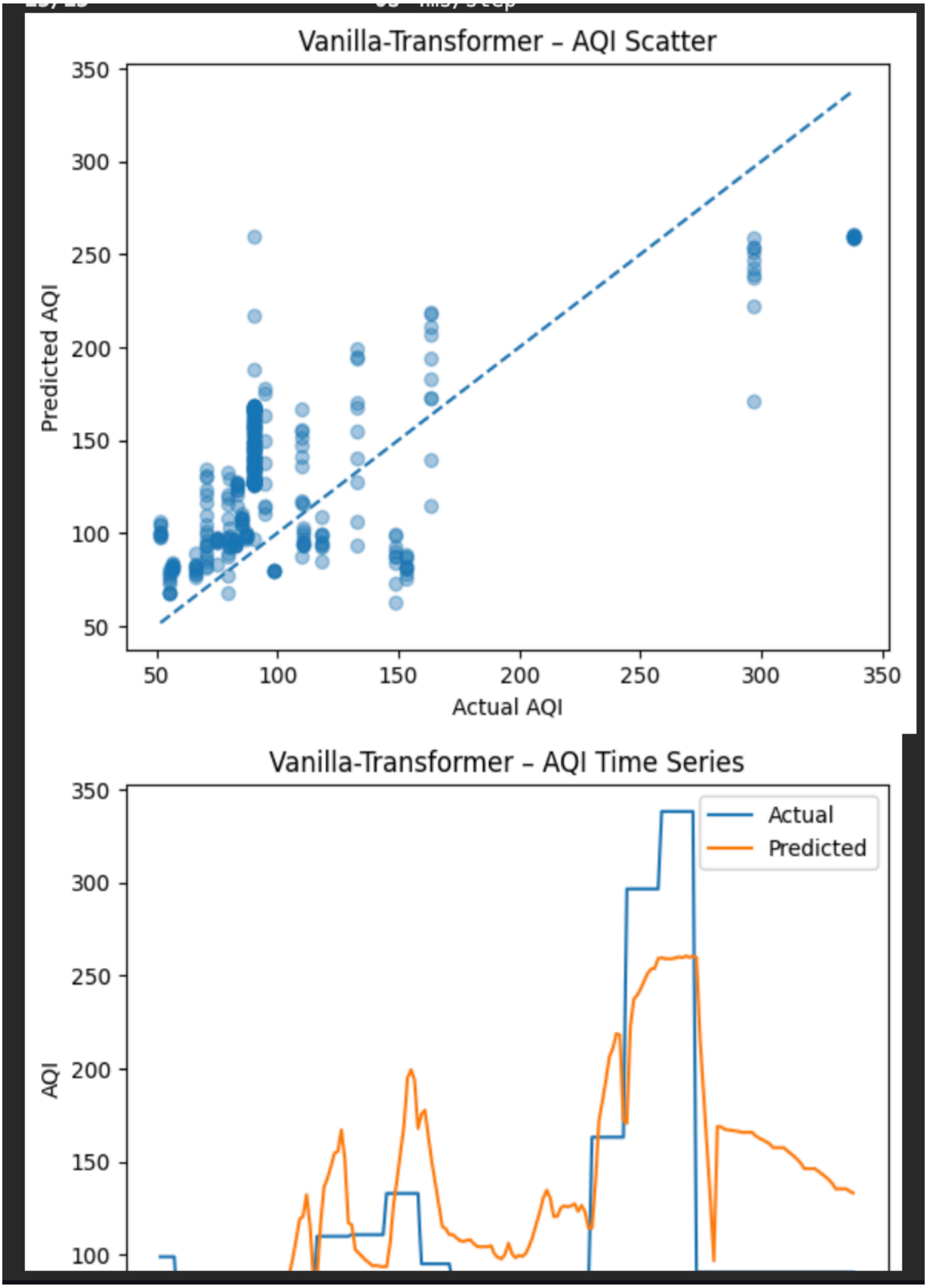
```
[ ]
def plot_residual_hist(residuals, title):
    plt.hist(residuals, bins=30)
    plt.title(title)
    plt.xlabel("Residual")
    plt.ylabel("Frequency")
    plt.show()
```

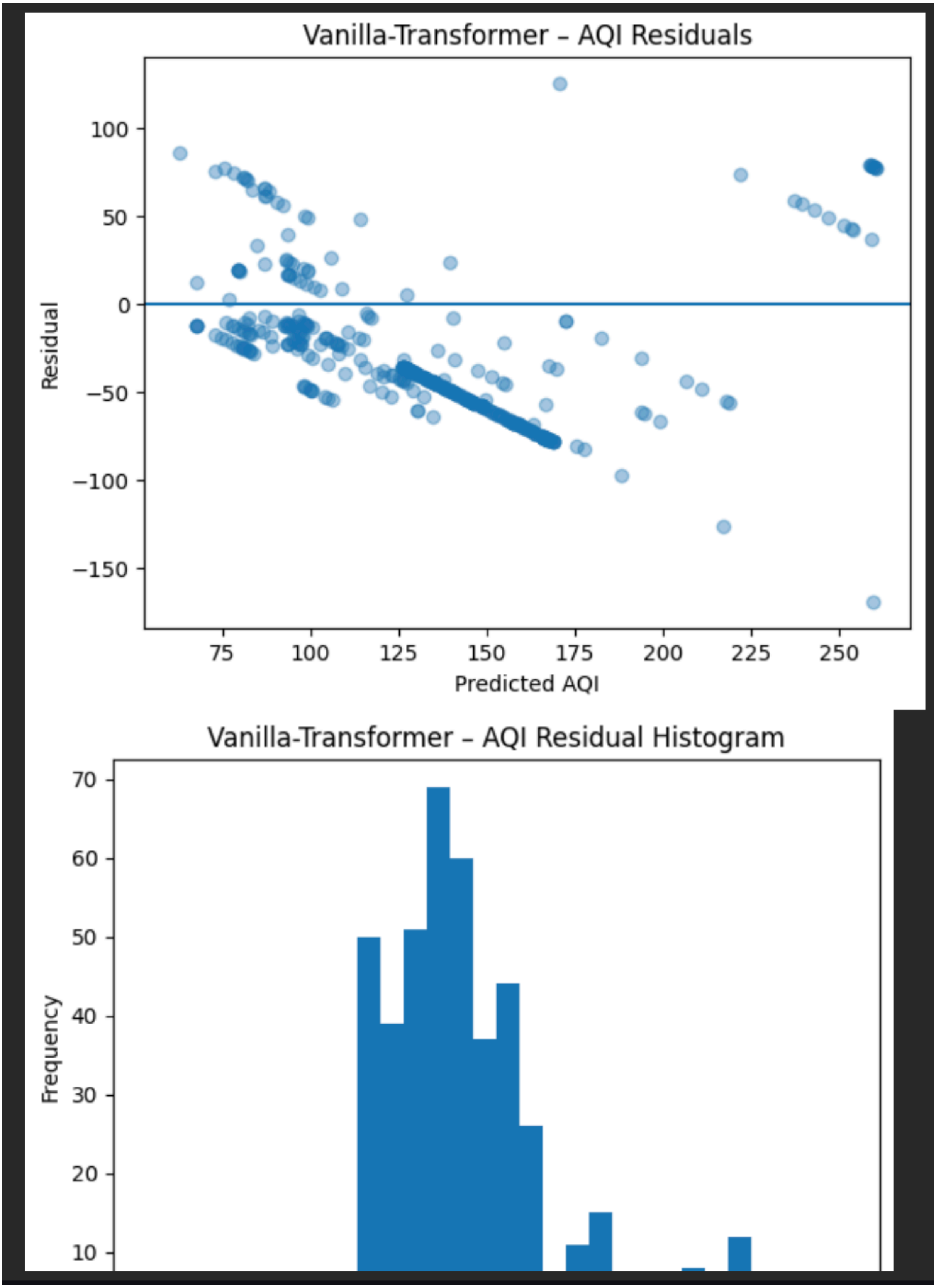
Q-Q Plot

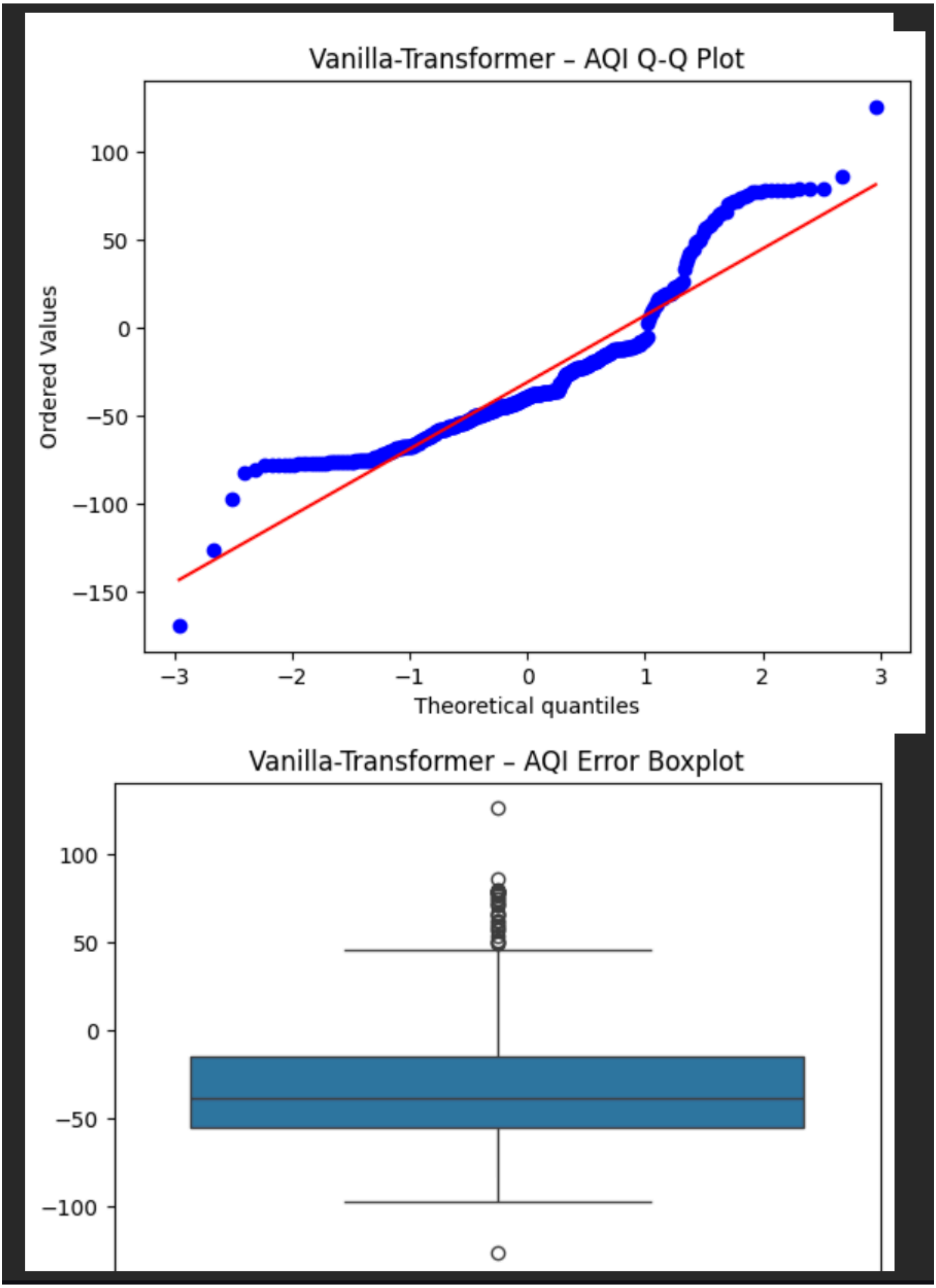
```
[ ]
def plot_qq(residuals, title):
    stats.probplot(residuals, plot=plt)
    plt.title(title)
    plt.show()
```

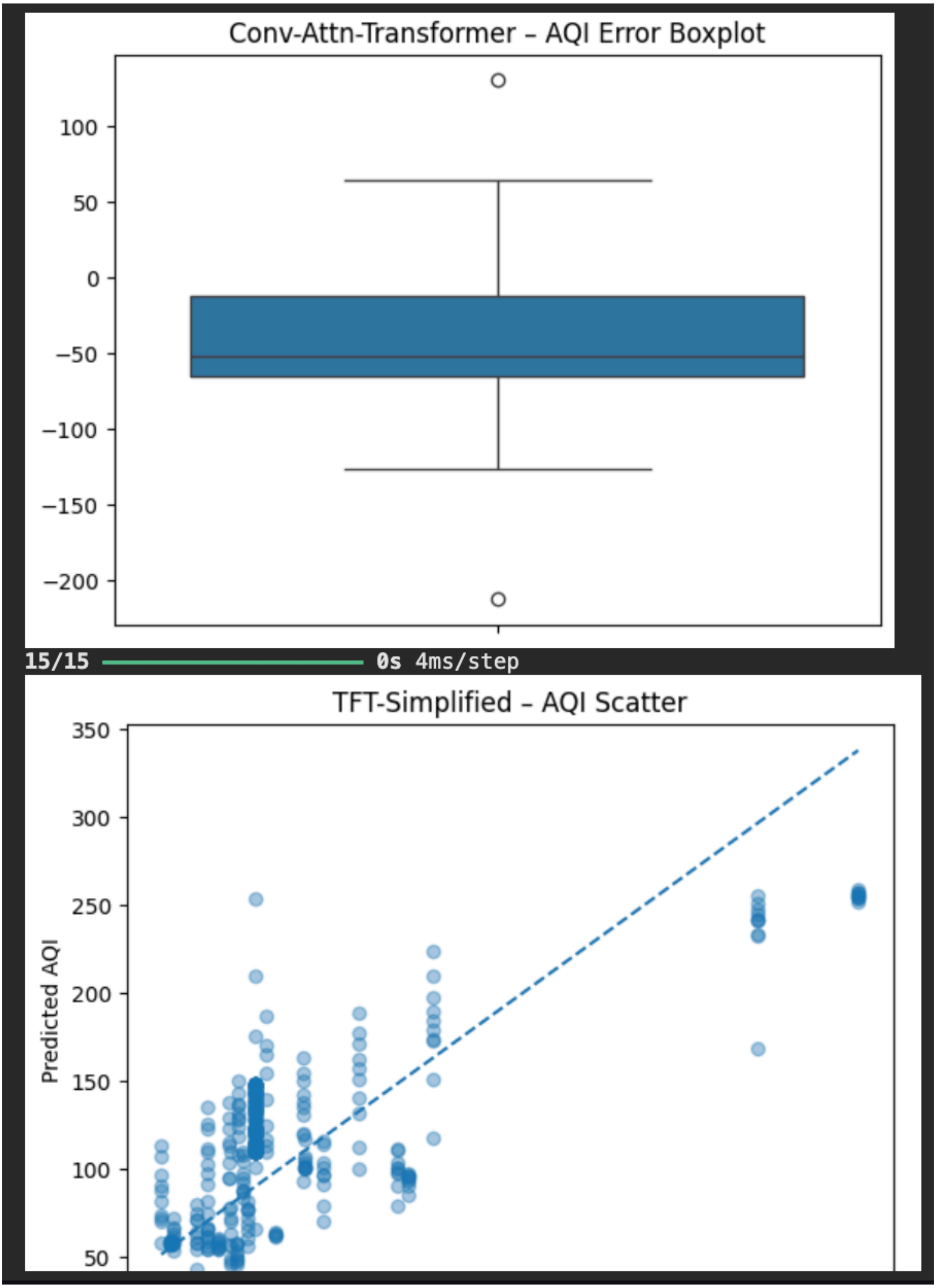
Box Plot (Error Distribution)

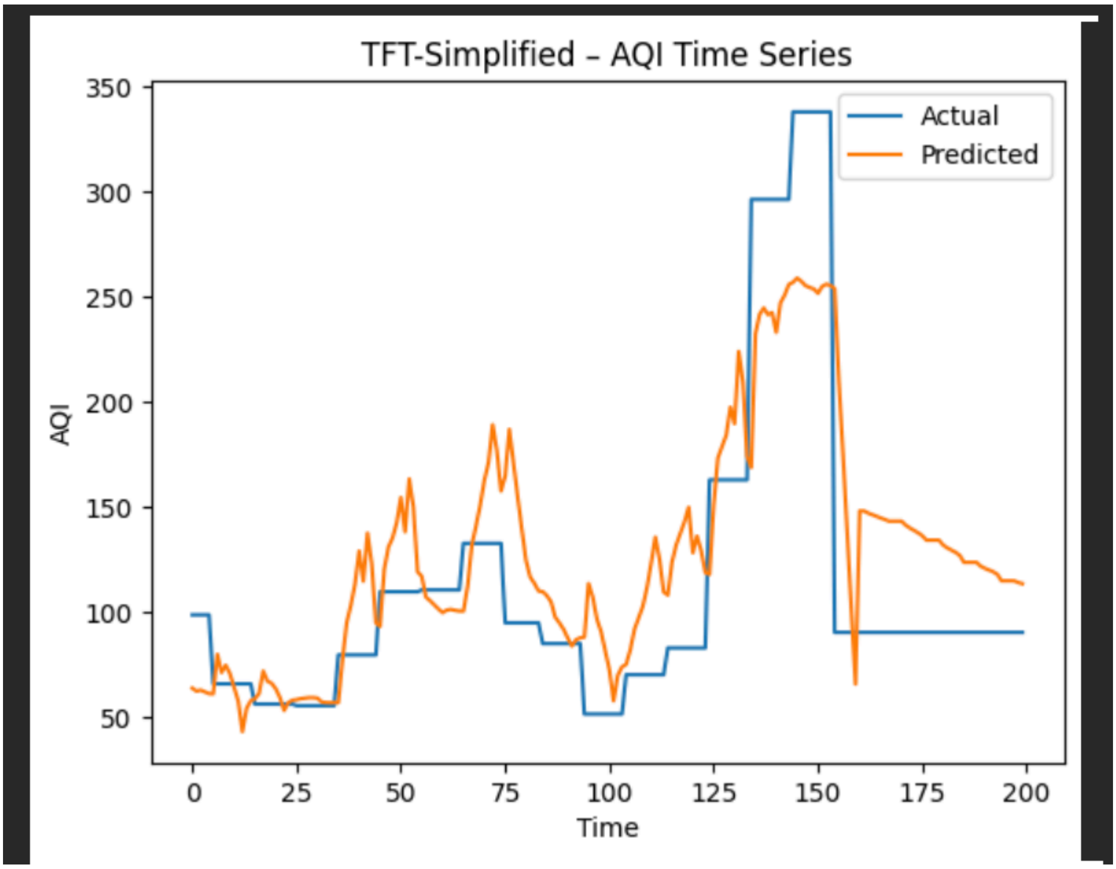
```
[ ]
def plot_error_box(residuals, title):
    sns.boxplot(residuals)
    plt.title(title)
    plt.show()
```

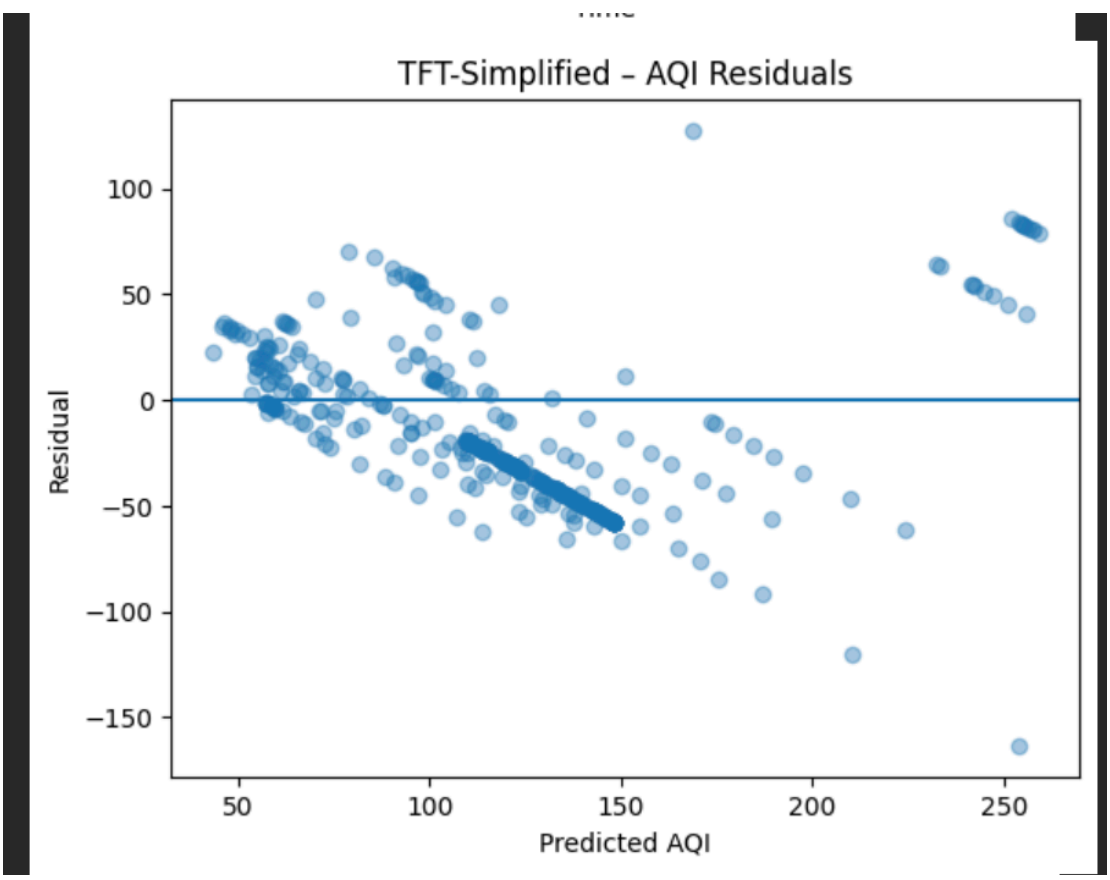

Vanilla-Transformer – AQI Scatter
Predicted AQI
Actual AQI
50
100
150
200
250
300
350
Vanilla-Transformer – AQI Time Series
AQI
Actual
Predicted


Vanilla-Transformer – AQI Residuals
Residual
Predicted AQI
Vanilla-Transformer – AQI Residual Histogram
Frequency


Vanilla-Transformer – AQI Q-Q Plot
Ordered Values
100
50
0
−50
−100
−150
−3
−2
−1
0
1
2
3
Theoretical quantiles
Vanilla-Transformer – AQI Error Boxplot
100
50
0
−50
−100


Conv-Attn-Transformer – AQI Error Boxplot
100
50
0
−50
−100
−150
−200
15/15 0s 4ms/step
TFT-Simplified – AQI Scatter
Predicted AQI
350
300
250
200
150
100
50


TFT-Simplified – AQI Time Series
Actual
Predicted
AQI
Time
350
300
250
200
150
100
50
0
25
50
75
100
125
150
175
200

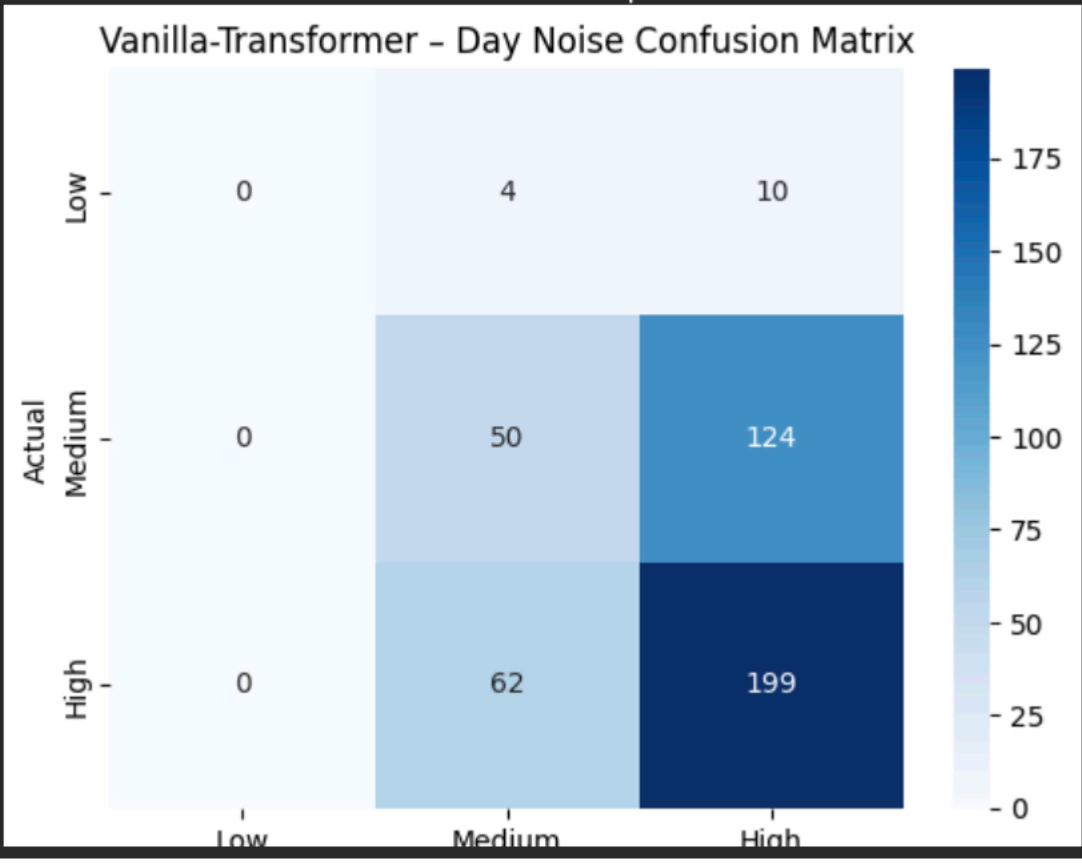

TFT-Simplified – AQI Residuals
Residual
Predicted AQI
100
50
0
−50
−100
−150
50
100
150
200
250

Generate ALL noise plots for ALL models

```
for name, model in trained_models.items():
    print(f"Noise plots for {name}")

    _, day_p, night_p = model.predict([X_test, city_test_ids])

    # ===== DAY NOISE =====
    plot_confusion(
        y_day_test,
        day_p,
        f"{name} — Day Noise Confusion Matrix"
    )

    plot_roc(
        y_day_test,
        day_p,
        f"{name} — Day Noise ROC"
    )

    plot_pr(
        y_day_test,
        day_p,
        f"{name} — Day Noise Precision—Recall"
    )

    plot_calibration(
        y_day_test,
        day_p,
        f"{name} — Day Noise Calibration"
    )

    # ===== NIGHT NOISE =====
    plot_confusion(
        y_night_test,
        night_p,
        f"{name} — Night Noise Confusion Matrix"
    )

    plot_roc(
        y_night_test,
        night_p,
```

```
    plot_roc(
        y_night_test,
        night_p,
        f"{name} – Night Noise ROC"
    )

    plot_pr(
        y_night_test,
        night_p,
        f"{name} – Night Noise Precision–Recall"
    )

    plot_calibration(
        y_night_test,
        night_p,
        f"{name} – Night Noise Calibration"
    )
```

```
... Noise plots for Vanilla–Transformer
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```



Generic attention probe builder

```
import tensorflow as tf
from tensorflow.keras.layers import MultiHeadAttention

def build_attention_probe_from_graph(model):
    mha_layer = None

    for layer in model.layers:
        if isinstance(layer, MultiHeadAttention):
            mha_layer = layer
            break

    if mha_layer is None:
        raise ValueError("No MultiHeadAttention layer found")

    # IMPORTANT: use the SAME tensor that was input to attention du
    mha_input = mha_layer.input[0]  # <- this is Conv1D output for

    # Re-run attention WITH scores
    _, attn_scores = mha_layer(
        mha_input, mha_input, return_attention_scores=True
    )

    return tf.keras.Model(
        inputs=model.input,
        outputs=attn_scores
    )
```

Build probes for all three models

Conv-attn-Transformer

```
attention_probes = {}

for name, model in trained_models.items():
    attention_probes[name] = build_attention_probe_from_graph(mode
```

Conv-attn-Transformer

```
attention_probes = {}

for name, model in trained_models.items():
    attention_probes[name] = build_attention_probe_from_graph(model
```

Generate heatmaps safely for ALL models

```
import numpy as np

def generate_recommendation(aqi_pred, day_prob, night_prob):
    alerts = []
    actions = []

    # AQI logic
    if aqi_pred > 300:
        alerts.append(f"CRITICAL: Hazardous Air Quality (AQI={aqi_p
        actions.append("Avoid outdoor activity; wear N95 mask")
    elif aqi_pred > 200:
        alerts.append(f"WARNING: Very Unhealthy Air (AQI={aqi_pred:
        actions.append("Close windows; limit outdoor exposure")

    # Day noise with confidence
    day_class = np.argmax(day_prob)
    day_conf = day_prob[day_class]

    if day_class == 2:
        alerts.append(
            f"ALERT: High Daytime Noise (confidence={day_conf:.2f})
        )
        actions.append("Use noise-canceling headphones")

    # Night noise with confidence
    night_class = np.argmax(night_prob)
    night_conf = night_prob[night_class]

    if night_class == 2:
        alerts.append(
```

Test the recommender on real predictions

```
vanilla_model = trained_models["Vanilla-Transformer"]

aqi_p, day_p, night_p = vanilla_model.predict(
    [X_test[:1], city_test_ids[:1]],
    verbose=0
)
generate_recommendation(
    aqi_p[0][0],
    day_p[0],
    night_p[0]
)
```

```
{'alerts': ['ADVISORY: High Night Noise (confidence=0.79)'],
 'recommendations': ['Use earplugs or white noise']}
```

Noise source

```
def infer_noise_source(day_prob, night_prob, hour):
    if hour >= 7 and hour <= 22:
        if np.argmax(day_prob) == 2:
            return "Likely sources: Traffic, Construction, Public Activity"
    else:
        if np.argmax(night_prob) == 2:
            return "Likely sources: Traffic, Industrial Operations, Night Transport"
    return "No dominant noise source detected"
```

```
source = infer_noise_source(day_p[0], night_p[0], hour=21)
```

RECOMMENDER VISUALS (DASHBOARD-STYLE)

Risk Score

```
risk_score = (
```

Risk Score

```python
risk_score = (
    (results_df["AQI_R2"] < 0.9).astype(int) +
    (results_df["Night_F1_weighted"] < 0.75).astype(int)
)
```

Calendar-style heatmap

```python
import seaborn as sns

risk_matrix = np.vstack([
    (results_df["AQI_R2"] < 0.9).astype(int),
    (results_df["AQI_MAPE"] > 8).astype(int),
    (results_df["Night_F1_weighted"] < 0.75).astype(int)
])

plt.figure(figsize=(10,4))
sns.heatmap(
    risk_matrix,
    cmap="Reds",
    annot=True,
    cbar=True,
    xticklabels=results_df["Model"],
    yticklabels=["AQI_R2", "AQI_MAPE", "Night_F1"]
)
plt.title("Multi-Metric Risk Sensitivity Heatmap")
plt.show()
```

Create AQI risk buckets

```python
import pandas as pd
import numpy as np

# AQI buckets based on CPCB / EPA style ranges
def aqi_bucket(aqi):
    if aqi <= 100:
        return "Low"
    elif aqi <= 200:
        return "Medium"
    else:
        return "High"
```

Compute residuals + buckets

```python
vanilla_model = trained_models["Vanilla-Transformer"]

aqi_pred, day_p, night_p = vanilla_model.predict(
    [X_test, city_test_ids],
    verbose=0
)
residuals = y_aqi_test - aqi_pred.flatten()

error_df = pd.DataFrame({
    "Actual_AQI": y_aqi_test,
    "Predicted_AQI": aqi_pred.flatten(),
    "Residual": residuals
})

error_df["AQI_Level"] = error_df["Actual_AQI"].apply(aqi_bucket)
```

FULL CALENDAR RISK HEATMAP

Create daily risk labels (AQI + Noise)

```
n = len(y_aqi_test)

dates = pd.date_range(
    start="2026-01-01",
    periods=n,
    freq="D"
)
```

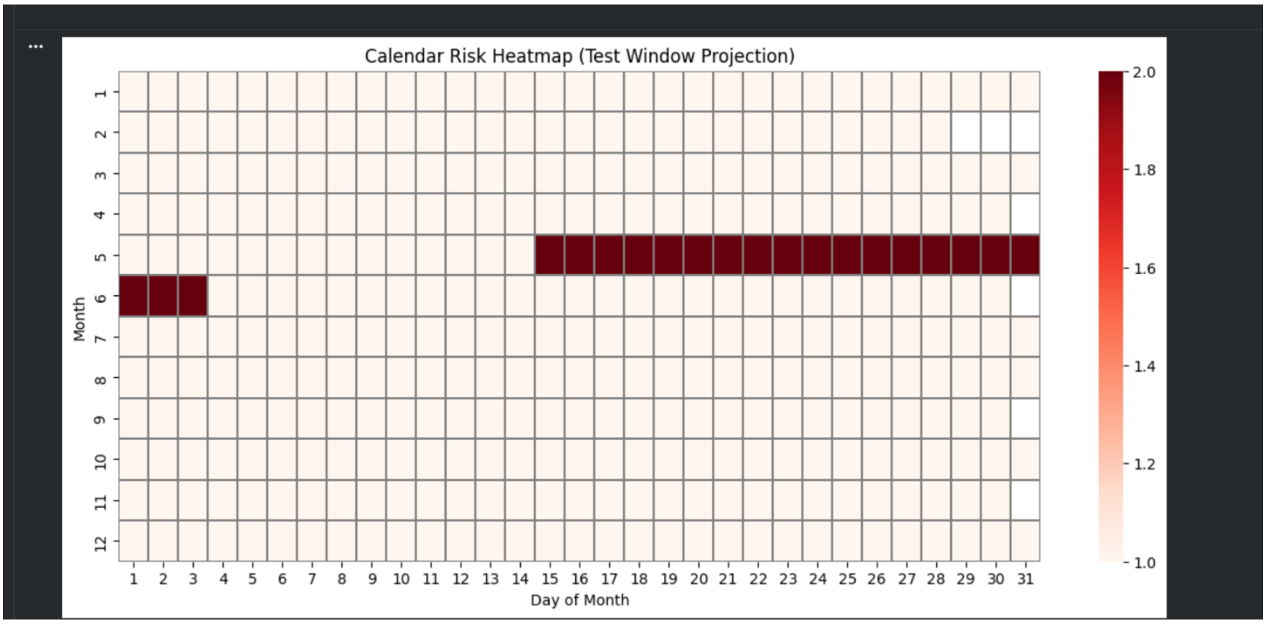
Build the calendar dataframe

```
calendar_df = pd.DataFrame({
    "Date": dates,
    "AQI": y_aqi_test,
    "Day_Noise": np.argmax(day_p, axis=1),
    "Night_Noise": np.argmax(night_p, axis=1)
})
```

Compute daily risk

```
def daily_risk(row):
    risk = 0
    if row["AQI"] > 200:
        risk += 1
    if row["Day_Noise"] == 2 or row["Night_Noise"] == 2:
        risk += 1
    return risk

calendar_df["Risk_Level"] = calendar_df.apply(daily_risk, axis=1)
```

```
calendar_df["Day"] = calendar_df["Date"].dt.day
calendar_df["Month"] = calendar_df["Date"].dt.month

pivot = calendar_df.pivot_table(
    index="Month",
    columns="Day",
    values="Risk_Level",
    aggfunc="max"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="Reds",
    linewidths=0.3,
    linecolor="gray",
    cbar=True
)
plt.title("Calendar Risk Heatmap (Test Window Projection)")
plt.xlabel("Day of Month")
plt.ylabel("Month")
plt.show()
```




```
calendar_df.head()
calendar_df.tail()
calendar_df["Risk_Level"].value_counts()
```

Risk_Level	count
1	425
2	20
0	4

dtype: int64

```
tft_model = trained_models["TFT-Simplified"]

tft_model.save("tft_simplified_model.keras")
```

```
from google.colab import files
files.download("tft_simplified_model.keras")
```

```
import joblib

joblib.dump(FEATURES, "features.pkl")
```

```
['features.pkl']
```

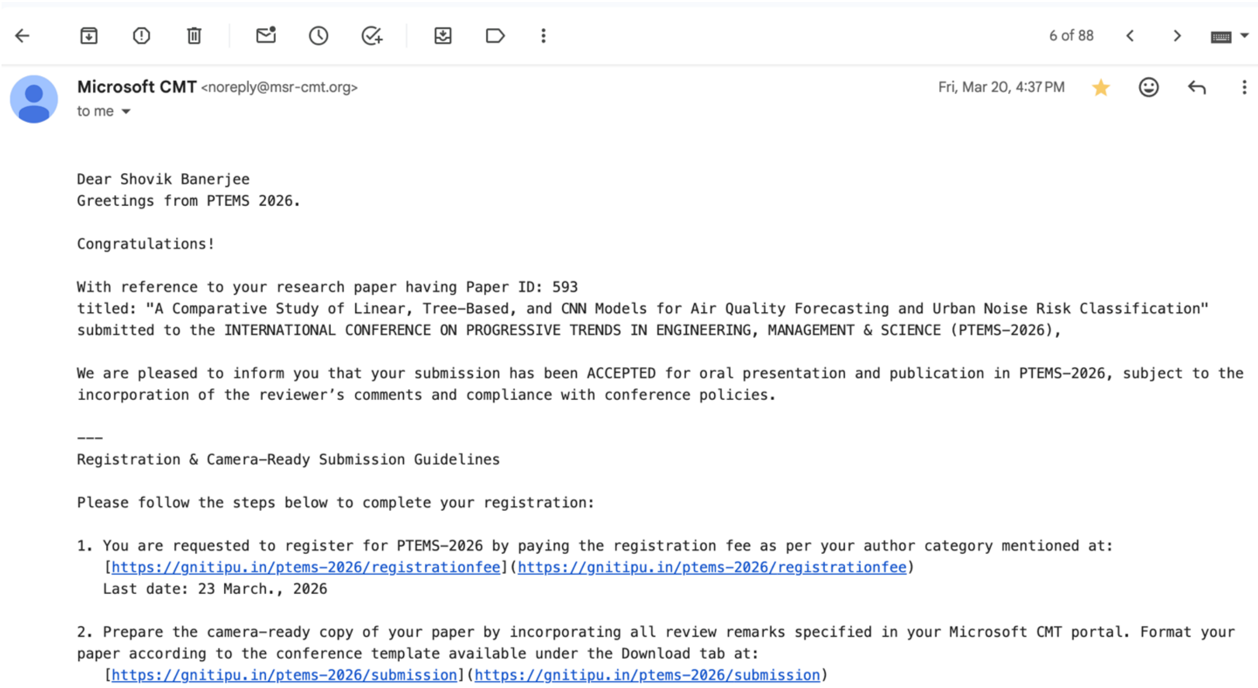
```
joblib.dump(6, "window.pkl")
```

```
['window.pkl']
```

APPENDIX B

CONFERENCE PRESENTATION

Our paper on **A Comparative Study of Linear, Tree-Based, and CNN Models for Air Quality Forecasting and Urban Noise Risk Classification** was presented at PTEMS 2026 conference held at GNIT-IPU. 200+ shortlisted teams presented their papers on various fields in the conference. Our paper got accepted as paper id : 593



6 of 88

Microsoft CMT <noreply@msr-cmt.org>
to me

Fri, Mar 20, 4:37 PM

Dear Shovik Banerjee
Greetings from PTEMS 2026.

Congratulations!

With reference to your research paper having Paper ID: 593
titled: "A Comparative Study of Linear, Tree-Based, and CNN Models for Air Quality Forecasting and Urban Noise Risk Classification"
submitted to the INTERNATIONAL CONFERENCE ON PROGRESSIVE TRENDS IN ENGINEERING, MANAGEMENT & SCIENCE (PTEMS-2026),

We are pleased to inform you that your submission has been ACCEPTED for oral presentation and publication in PTEMS-2026, subject to the incorporation of the reviewer's comments and compliance with conference policies.

Registration & Camera-Ready Submission Guidelines

Please follow the steps below to complete your registration:

1. You are requested to register for PTEMS-2026 by paying the registration fee as per your author category mentioned at:
   https://gnitipu.in/ptems-2026/registrationfee
   Last date: 23 March., 2026

2. Prepare the camera-ready copy of your paper by incorporating all review remarks specified in your Microsoft CMT portal. Format your paper according to the conference template available under the Download tab at:
   https://gnitipu.in/ptems-2026/submission

Figure B.1: PTEMS 2026 Acceptance

On presenting the paper in this international conference held at Greater Noida Institute of Technology, Noida campus, we received positive remarks and suggestion from the judging panel.

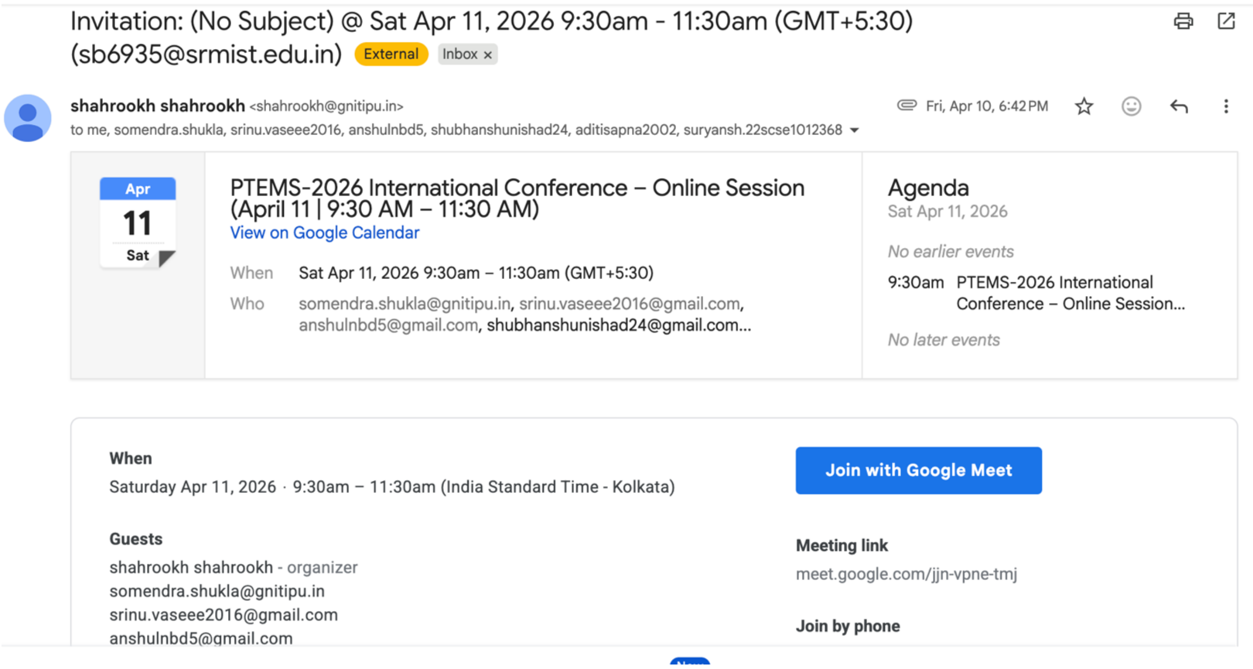


Figure B.2 Presentation Invite

Figure B.3: Certificate of Presentation

APPENDIX C

PUBLICATION DETAILS

We submitted our research paper for publication at Springer Nature and the paper is under peer review as of now. Publication status is attached in figure B.1

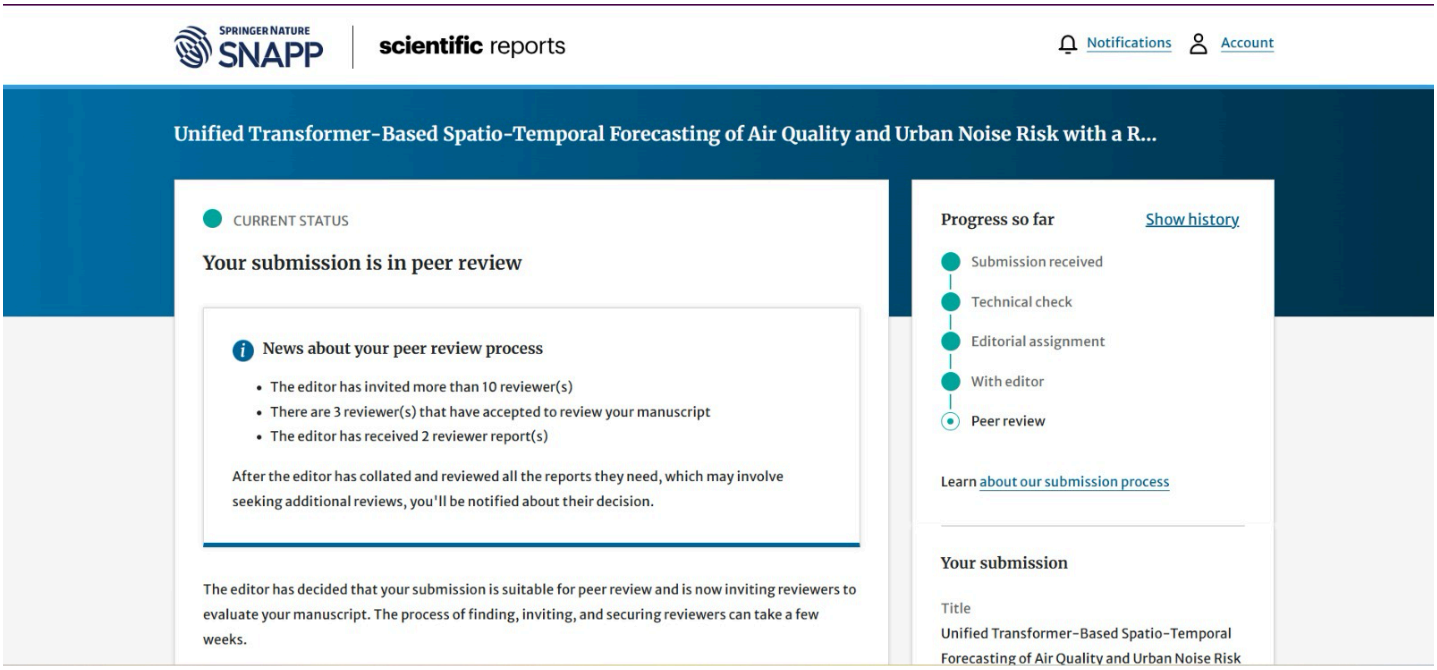


Figure C.1: Publication Status

APPENDIX D

PLAGIARISM REPORT

Project Report.pdf

Institute of Aeronautical Engineering (IARE)

Document Details

Submission ID

trn:oid:::3618:137096941

Submission Date

Apr 29, 2026, 9:47 PM GMT+5:30

Download Date

Apr 29, 2026, 9:50 PM GMT+5:30

File Name

Project Report.pdf

File Size

22.8 MB

67 Pages

11,067 Words

66,175 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 8 words)
- Abstract
- Methods and Materials

Match Groups

- **24** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2%	Internet sources
1%	Publications
0%	Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **24** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
1% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
	www.mdpi.com	<1%
2	Internet	
	flsr.de	<1%
3	Internet	
	arxiv.org	<1%
4	Internet	
	s40026.pcdn.co	<1%
5	Internet	
	www.holdfast.sa.gov.au	<1%
6	Internet	
	escc.uth.gr	<1%
7	Internet	
	livresnumeriques.brouillondeculture.fr	<1%
8	Internet	
	www.frontiersin.org	<1%
9	Publication	
	Dhirendra Kumar Shukla, Shabir Ali, Sandhya Sharma. "Artificial Intelligence and ...	<1%
10	Publication	
	Sukhpreet Kaur, Amanpreet Kaur, Manish Kumar. "Recent Advances in Computati...	<1%